

ELOAH KAORI NOGUCHI

**LAKEHOUSE: ANÁLISE DE DESEMPENHO E COMPARAÇÃO DE
SOLUÇÕES**

**Monografia apresentada ao Programa de
Educação Continuada da Escola
Politécnica da Universidade de São
Paulo, para obtenção do título de
Especialista, pelo Programa de Pós-
Graduação em Engenharia de Dados e
Big Data.**

SÃO PAULO

2024

ELOAH KAORI NOGUCHI

**LAKEHOUSE: ANÁLISE DE DESEMPENHO E COMPARAÇÃO DE
SOLUÇÕES**

**Monografia apresentada ao Programa de
Educação Continuada da Escola
Politécnica da Universidade de São
Paulo, para obtenção do título de
Especialista, pelo Programa de Pós-
Graduação em Engenharia de Dados e
Big Data.**

**Área de concentração: Tecnologia da
Informação – Engenharia/ Tecnologia/
Gestão**

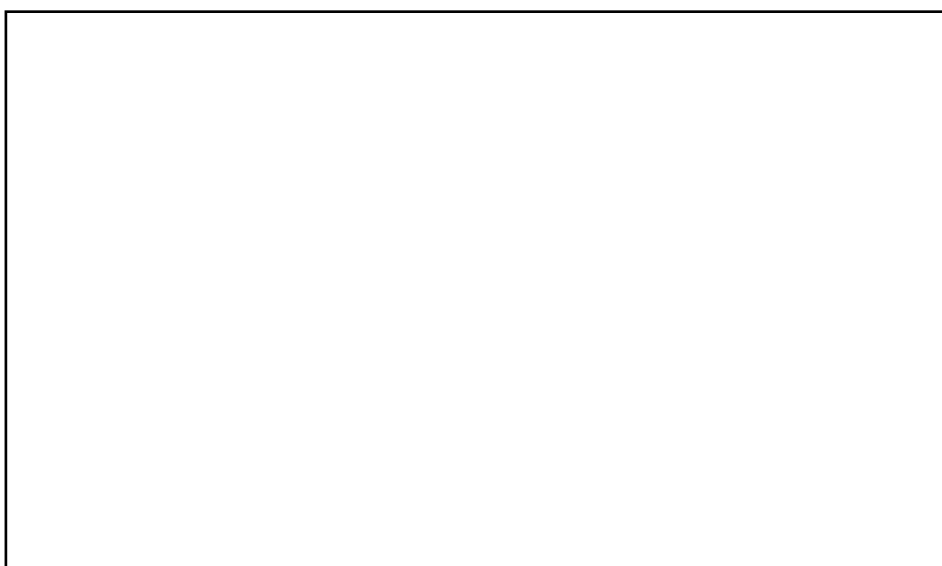
**Orientador: Prof. Me. Leandro Mendes
Ferreira**

SÃO PAULO

2024

Autorizo a reprodução e divulgação total ou parcial deste trabalho, por qualquer meio convencional ou eletrônico, para fins de estudo e pesquisa, desde que citada a fonte.

Catálogo-na-publicação



AGRADECIMENTOS

Gostaria de expressar minha profunda gratidão aos meus pais, cujo apoio tem sido fundamental em toda minha jornada. Desde o início da minha trajetória, eles sempre me incentivaram e motivaram aos meus estudos, o que me permitiu chegar até aqui. Sem o apoio e a dedicação deles, este trabalho não seria possível.

Agradeço também à minha irmã, que tem sido uma grande fonte de inspiração e um suporte essencial em muitos momentos dessa trajetória. Dedico este trabalho a mim mesma, pelo esforço incansável, pelas horas de dedicação e pela constante lembrança dos meus objetivos profissionais.

Agradeço ao meu orientador, Prof. Me. Leandro Mendes Ferreira, pela dedicação e pronta atenção sempre que precisei de apoio ou direção. Suas contribuições foram essenciais para o desenvolvimento deste trabalho.

Agradeço à USP e ao programa PECE Poli pela oportunidade de cursar este programa de pós-graduação.

CURSO ENGENHARIA DE DADOS E BIG DATA

Coord.: Profa. Dra. Solange N. Alves de Souza

Vice-Coord.: Profa. Dra. Anarosa Alves Franco Brandão

Perspectivas profissionais alcançadas com o curso:

[Com a conclusão do curso pude ampliar minha visão sobre dados, tanto no aspecto teórico quanto prático. A oportunidade de conhecer profissionais da mesma área foi enriquecedora, pois possibilitou a troca de experiências e o aprendizado com pessoas que enfrentam desafios similares no dia a dia, o que ampliou minha rede de contatos e abriu portas para novas colaborações.]

Além disso, o curso me proporcionou o aprendizado de novas ferramentas e tecnologias, essenciais para a atuação em ambientes de dados que estão cada vez mais complexos e dinâmicos. O conhecimento adquirido sobre essas ferramentas me permitiu aumentar minha confiança profissional e participação em reuniões e projetos.]

RESUMO

Com o aumento da necessidade de dados nas organizações, surgiram abordagens como *Data Warehouse* e *Data Lake* para armazenamento e gerenciamento de grandes volumes de dados. No entanto, essas arquiteturas de gestão de dados enfrentaram limitações quanto à flexibilidade, escalabilidade e tratamento de dados semiestruturados e não estruturados. Em resposta, a arquitetura híbrida *Data Lakehouse* foi criada, unificando o gerenciamento estruturado do *Data Warehouse* com a escalabilidade e o baixo custo do *Data Lake*.

Diante do panorama de desafios relacionados à otimização da implementação de uma arquitetura híbrida, emergiram no mercado soluções tecnológicas como *Delta Lake*, *Apache Hudi* e *Apache Iceberg*. Este trabalho tem como objetivo analisar e comparar essas três tecnologias em relação a tempo de execução, eficiência e contribuir com embasamento para tomada de decisão a partir de uma dessas tecnologia que melhor atende a necessidade de flexibilidade, escalabilidade e desempenho.

Essa análise foi executada na nuvem AWS, utilizando EMR para executar o processo de ingestão e atualização dos dados, S3 para armazená-los e o Athena para consumir e realizar consultas. Além disso, houve uma leitura com *PrestoDB*, onde foi comparado com o desempenho do Athena. Essa massa de dados teste foi gerada via SSB (*Star Schema Benchmark*) que é utilizado para simular um ambiente de *Data Warehouse* com fatos e dimensões, permitindo avaliar consultas complexas e seu desempenho.

Palavras-chave: Apache Hudi, Apache Iceberg, Delta Lake, Data Lake, Data Lakehouse.

ABSTRACT

With organizations' increasing need for data, approaches such as Data Warehouses and Data Lakes emerged for storing and managing large volumes of data. However, these data management architectures faced limitations regarding flexibility, scalability, and the handling of semi-structured and unstructured data. In response, the hybrid Data Lakehouse architecture was created, combining the structured management of Data Warehouses with the scalability and low cost of Data Lakes.

In light of the challenges related to optimizing the implementation of a hybrid architecture, technological solutions such as Delta Lake, Apache Hudi, and Apache Iceberg emerged in the market. This study aims to analyze and compare these three technologies in terms of performance time and efficiency, and to contribute to decision-making based on which technologies best meets the needs of flexibility, scalability, and performance.

This analysis was conducted in the AWS cloud, using EMR to run the data ingestion and update process, S3 for data storage, and Athena for consuming and querying the data. Additionally, PrestoDB was compared to evaluate its performance against Athena. The test data set was generated via the Star Schema Benchmark (SSB), which simulates a Data Warehouse environment with facts and dimensions, allowing for the evaluation of complex queries and their performance.

Keywords: Apache Hudi, Apache Iceberg, Delta Lake, Data Lake, Data Lakehouse.

LISTA DE FIGURAS

Figura 1 - Esquema do <i>Star Schema Benchmark</i>	25
Figura 2 - Arquitetura de Spark.....	27
Figura 3 - Arquitetura Proposta	33
Figura 4 - Resultado da tabela config_df.....	34
Figura 5 - Resultado de desempenho de Escrita.	38
Figura 6 - Resultado de desempenho de Atualização.....	39
Figura 7 - Resultado de desempenho de Leitura após Escrita.....	40
Figura 8 - Resultado de desempenho de Leitura após Atualização.	40
Figura 9 - Resultado do desempenho das 13 consultas do Delta Lake da Leitura após escrita com Athena.....	42
Figura 10 - Resultado do desempenho das 13 consultas do Delta Lake da Leitura após escrita com PrestoDB.....	42
Figura 11 - Resultado do desempenho das 13 consultas do Apache Hudi da Leitura após escrita com Athena.....	43
Figura 12 - Resultado desempenho das 13 consultas do Delta Lake da Leitura após escrita com PrestoDB.	42
Figura 13 - Resultado desempenho das 13 consultas do Iceberg da Leitura após escrita com Athena.	42
Figura 14 - Resultado desempenho das 13 consultas do Delta Lake da Leitura após a atualização lidos com o Athena.	44
Figura 15 - Resultado desempenho das 13 consultas do Delta Lake da Leitura após atualização lidos com PrestoDB.	45

Figura 16 - Resultado desempenho das 13 consultas do Hudi da Leitura após atualizações lidos com Athena.....	45
Figura 17 - Resultado desempenho das 13 consultas do Hudi da Leitura após atualizações lidos com o PrestoDB.....	46
Figura 18 - Resultado desempenho das 13 consultas do Iceberg da Leitura após atualizações lidos com Athena.....	42
Figura 19 - Resultado desempenho de Leitura após escrita com consulta simples..	47
Figura 20 - Resultado desempenho de Leitura após atualização com consulta simples.....	48

LISTA DE TABELAS

Tabela 1 - Comparação arquitetura de dados	19
--	----

LISTA DE ABREVIATÖES

ACID – Atomicidade, Consistência, Isolamento e Durabilidade

APIs – Application Programming Interface

AWS – Amazon Web Services

DL – Data Lake

DLH – Data Lakehouse

DM – Data Marts

DW – Data Warehouse

EBS – Elastic Block Store

EMR – Elastic MapReduce

ETL – Extract, Transforming, Loading

OLAP – Online Analytical Processing

OLTP – Online Transactional Processing

S3 – Simple Storage Service

SSB – Star Schema Benchmark

SUMÁRIO

1	INTRODUÇÃO	12
1.1	Motivação.....	13
1.2	Objetivo.....	13
1.3	Justificativa	14
1.4	Metodologia	14
2	FUNDAMENTAÇÃO TEÓRICA	15
2.1	<i>Big Data</i>	15
2.2	<i>Data Warehouse</i>	16
2.3	<i>Data Lake</i>	18
2.4	<i>Data Lakehouse</i>	19
2.5	<i>Delta Lake</i>	21
2.6	<i>Apache Hudi</i>	22
2.7	<i>Apache Iceberg</i>	23
2.8	<i>PrestoDB</i>	24
2.9	<i>Star Schema Benchmark</i>	24
2.10	<i>Apache Spark</i>	27
3	EXPERIMENTO.....	29
3.1	Ambiente de desenvolvimento	29
3.2	Arquitetura Proposta	33
3.3	Detalhamento do Experimento.....	34
3.3.1	Utilização do Delta Lake.....	36
3.3.2	Utilização do Apache Hudi.....	37
3.3.3	Utilização do Apache Iceberg	38

4	RESULTADOS	39
5	CONCLUSÃO.....	51
5.1	Contribuições do trabalho.....	52
5.2	Trabalhos futuros.....	53
	REFERÊNCIAS BIBLIOGRÁFICA	54
	APÊNDICE A	57
A.1.	Consultas disponibilizadas pelo SSB.....	57
A.2.	Dados granulares dos resultados.....	60
A.2.1.	Escrita	60
A.2.2.	Atualização	60
A.2.3.	Leitura após a escrita.....	60
A.2.4.	Leitura após a atualização.....	64
A.3.	Consultas simples.....	68
A.3.1.	Após a escrita.....	68
A.3.2.	Após a atualização	68

1 INTRODUÇÃO

Com o aumento da volumetria de dados nas organizações, a necessidade de armazenamento e gerenciamento e a utilização deles para necessidades competitivas perante o mercado. Surgiram várias abordagens para solucionar esse problema como o *Data Warehouse* (DW), que foi construído para armazenar dados estruturados permitindo uma análise eficiente devido ao esquema fixo e facilitando a geração de relatórios para as organizações.

A definição de DW segundo Inmon (2005) é que se trata de uma coleção de dados, orientada a assuntos, integrada, não volátil e que varia ao longo do tempo, com o propósito de apoiar as decisões gerenciais. Porém, com a utilização do DW, notou-se a falta de flexibilidade para lidar com dados semiestruturados e não estruturados, além da dificuldade em escalar para volumes massivos de dados e a dependência de modelos rígidos de esquema.

Já o *Data Lake* (DL), segundo Fang (2015), é um grande repositório de dados que tem como objetivo melhorar a captura, o refinamento, o armazenamento e a exploração de dados dentro da organização. O DL contém dados estruturados ou semiestruturados e é considerado uma evolução da arquitetura de dados existente devido ao baixo custo e a facilidade em escalar, adaptando-se a volumetria. No entanto, há algumas preocupações na arquitetura que precisam ser ressaltadas, como a falta de definição inicial do esquema de dados. Com a ideia de coletar dados e entender a necessidade deles depois, as organizações precisam criar esses esquemas de qualquer maneira para que esses dados sejam analisados.

Outra preocupação do DL é a inconsistência dos dados na primeira camada pois não existe tratamento e processamento para a ingestão dos dados (Sonam Ramchand e Tariq Mahmood, 2022) tornando-os não confiáveis para a análise e acrescentando uma etapa a mais de confiabilidade.

Em resposta a esses pontos, a arquitetura híbrida *Data Lakehouse* foi criada, unificando o gerenciamento estruturado do DW com a escalabilidade e o baixo custo do DL. O *Data Lakehouse* surge com uma proposta de oferecer baixo custo de

armazenamento e flexibilidade para armazenar e processar tanto dados estruturados quanto não estruturados. Essa abordagem possibilita consultas analíticas estruturadas e análises avançadas, permitindo uma solução completa para atender às crescentes demandas de dados em tempo real e *big data*.

Diante do panorama de desafios relacionados à otimização da implementação de uma arquitetura híbrida de *Data Lakehouse*, emergiram no mercado soluções tecnológicas como *Delta Lake*, *Apache Hudi* e *Apache Iceberg*. Elas foram desenvolvidas para suprir as limitações de consistência, desempenho e governança de dados em grandes volumes.

1.1 Motivação

As decisões de arquitetura e da tecnologia mais adequada a organização pode ter um impacto diretamente no desempenho dos processos de ingestão e consumo de dados, englobando desde o processamento até mesmo a qualidade e integridade dos dados inseridos.

Com isso, é importante ter ciência de como essas tecnologias se comportam em ambientes diversos com diferentes volumetrias. A realização de um *benchmark* entre o *Apache Hudi*, *Delta Lake* e *Apache Iceberg* permitirá a identificação de qual tecnologia se destaca em determinadas situações que podem beneficiar as organizações, oferecendo uma base para decisões na arquitetura de dados.

1.2 Objetivo

Esse trabalho tem como objetivo analisar e comparar o desempenho de três tecnologias utilizadas no *Data Lakehouse*: *Delta Lake*, *Apache Hudi* e *Apache Iceberg*. A pesquisa foca em três aspectos importantes para estas arquiteturas de dados: leitura, escrita e atualização dos dados.

1.3 Justificativa

Algumas pesquisas realizaram a comparação do tempo de escrita, leitura e atualização do dado, como por exemplo, Jain et al. (2023). Eles utilizaram uma outra arquitetura para desenvolvimento. No entanto, a proposta desse trabalho é fazer a mesma comparação, mas utilizando o *Delta Lake*, *Apache Hudi* e *Apache Iceberg*. O resultado desse trabalho adiciona mais embasamento para essa escolha entre as tecnologias e traz resultados utilizando outras ferramentas como o PrestoDB, massa de dados com diferentes volumetrias e principalmente analisar o tempo de atualização dos dados.

1.4 Metodologia

Com o objetivo de comparar o desempenho referente ao tempo de leitura, escrita e atualização dos dados, a metodologia escolhida foi um *benchmark* comparando as tecnologias *Delta Lake*, *Apache Hudi* e *Apache Iceberg*. Para o desenvolvimento, foi utilizado o *Star Schema Benchmark* (SSB) que foi criado para medir o tempo de desempenho ao executar consultas no modelo de *Star Schema* (Dehdouh, Boussaid e Bentayeb, 2014).

2 FUNDAMENTAÇÃO TEÓRICA

2.1 *Big Data*

Big Data usualmente é um termo que se refere a uma grande volumetria de dados que inclui diferentes formatos de dados: estruturados, semiestruturados e não estruturados (Oussous et al. 2018). Devido a quantidade de dados ter crescido exponencialmente ao longo dos últimos anos, gerou-se vários desafios de coleta, armazenamento e processamento de todos esses dados para as organizações, levando-as a priorizar e entender como esses dados as ajudariam a se destacar no mercado que se tornou cada vez mais competitivo.

Tradicionalmente, existem três características que podem descrever melhor o termo *Big Data*, elas são chamadas de 3Vs (Volume, Velocidade e Variedade). Gandomi e Haider (2015) apresentaram a seguinte proposta com base nos 3Vs:

- **Volume:** Refere-se à quantidade de dados em uma escala de terabytes, petabytes e assim por diante. É um termo relativo que pode variar com o tempo e o tipo de dados pois o que é considerado *Big Data* hoje pode não ser no futuro devido as melhorias de armazenamento, processamento e manipulação.
- **Velocidade:** Refere-se à velocidade que a informação deve ser extraída e processada para geração de percepções por período (Oussous et al. 2018). A velocidade que esses dados são processados é fundamental para o funcionamento de algumas áreas como por exemplo detecção de fraude em organizações financeiras ou acompanhamento em tempo real de dispositivos médicos.
- **Variedade:** Refere-se a variedade estrutural dos dados que são eles: estruturados, semiestruturados e não estruturados. Os dados estruturados são os dados encontrados em banco de dados relacionais e em planilhas, que são organizados e possuem esquemas rígidos. Já os semiestruturados podem ser os arquivos no formato JSON, logs ou e-mails, eles não possuem uma estrutura tão

rígida quanto os estruturados, porém tem uma estrutura mais básica que permite a leitura para as máquinas. E por fim os dados não estruturados que não tem uma estrutura definida, tornando-os mais complexos de processar como por exemplo dados de multimídia (imagens, áudios e vídeos) e *pdf* que possuem um formato livre.

2.2 Data Warehouse

Um *Data Warehouse* (DW) é um repositório para todos os dados coletados por uma organização com diversas fontes de dados. A condição de um DW pode ser física ou lógica, já que o DW é composto por uma coleção de dados orientada a assunto, integrada, variante no tempo e não volátil, em suporte ao processo de tomada de decisão gerencial (Kumar e Kavita 2019).

Em geral, o DW é implementado em banco de dados relacional e mantido separadamente dos bancos de dados operacionais de uma organização. O DW suporta o processamento analítico *on-line* (OLAP (*Online Analytical Processing*) – acrônimo em inglês), cujos requisitos funcionais e de desempenho são bastante diferentes dos aplicativos de processamento de transações *on-line* (OLTP (*Online Transactional Processing*) – acrônimo em inglês) tradicionalmente suportados pelos bancos de dados operacionais (Chaudhuri e Dayal 1997). Adicionalmente, técnicas usadas para modelagens tradicionais bancos de dados operacionais, como modelagem relacional, são inadequadas para projetar DWs. Por essa razão, existem modelagens de dados especificamente para o *design* de DW, sendo a modelagem dimensional a abordagem predominante adotada para o *design* de DW (Moody e Kortink 2000).

Segundo Yessad e Labiod (2016), a implementação de um DW possui duas abordagens principais que são discutidas e detalhadas por Inmon(2005) e Kimball(2013), na visão de Inmon, a estrutura de DW possui vários dados organizados por assuntos que podem variar de organização para organização, mantendo o histórico de todas as transações e que não podem ser alterados. Já na

visão de Kimball, ele defende a ideia de que o DW é uma cópia do transacional e está estruturado para atender as tomadas de decisão.

A principal diferença entre a visão dos dois autores é como é estruturado o armazenamento e a integração dos dados. Inmon defende a criação de um DW centralizado, garantindo uma consistência de dados e disponibilização dos mesmos para toda a organização e Kimball defende o DW com *Data Marts* (DMs). DMs são subconjuntos de dados, que atendem diretamente as necessidades das análises, com foco em disponibilizar os dados mais rápido possível para os analistas de dados.

Yessad e Labiod (2016) afirmam que a abordagem de Inmon é recomendada para organizações que possuem mais de uma área de inteligência de negócio que irá consumir os dados e essas análises não estão definidas. Enquanto a abordagem de Kimball é recomendada onde as análises e objetivos estão bem definidos e procura-se um alto desempenho nas consultas para o usuário final.

O DW de Inmon (2005) propõe uma modelagem com o *Star Schema*, que organiza os dados com uma tabela central de fatos que contém os dados quantitativos, como vendas ou lucros, e tabelas de dimensões ao redor, que descrevem esses dados, como tempo, produto, cliente e local. Esse formato facilita a realização de consultas rápidas e a criação de relatórios analíticos detalhados, ideal para organizações que possuem as decisões de negócios baseado em dados, onde a clareza e a agilidade na análise de dados são essenciais.

Apesar dessas duas abordagens, de acordo com Raslan e Calazans (2014), os pontos positivos em adotar o DW como estrutura são: a possibilidade de integração sólida dos dados; suporte à análise gerencial estratégica; organização de dados provenientes de fontes internas e externas; e a garantia de consistência e segurança das informações para tomada de decisões. E os pontos negativos são: o elevado custo de implementação e manutenção; o risco de obsolescência rápida devido às constantes inovações tecnológicas; a latência de dados gerada pelo processo de extração, transformação e carregamento (ETL – *Extract, Transforming, Loading*), além da necessidade de apoio contínuo da alta gerência para garantir o sucesso do

projeto. Além disso, os DWs tradicionais enfrentam desafios para lidar com dados não estruturados, o que pode limitar a flexibilidade e adaptabilidade do sistema às necessidades das companhias

2.3 Data Lake

Segundo Singh et al. (2022), o DL é um repositório centralizado arquitetado para armazenar grandes volumes de dados em seu formato original, sem a necessidade imediata de estruturação. Ao contrário dos DWs, que organizam os dados de maneira estruturada e hierárquica, um DL permite armazenar dados oriundos de várias fontes como redes sociais, logs de servidores, arquivos de áudio, vídeo e texto em seu estado bruto até que sejam necessários para processamento e análise. Segundo Zhang et al. (2023), o conceito de DL foi proposto nas organizações para mitigar as limitações do DW quando se tratam de gerenciamento de dados, controle de versão e acesso.

O DL adota uma abordagem em camadas, permitindo uma organização flexível que acomoda uma variedade de formatos, como dados estruturados, semiestruturados e não estruturados. Essa flexibilidade é crucial para lidar com o crescente volume de dados gerados pelas organizações, além de proporcionar uma escalabilidade eficiente da infraestrutura necessária para suportar essa volumetria (Singh et al. 2022). Segundo Singh et al. (2022), o DL pode ser organizado em três camadas que possuem funções e organizações distintas do dado:

1. A primeira camada é chamada de *Raw Data Zone* que tem como objetivo armazenar os dados em seu estado bruto, igual a origem, sem qualquer transformação e tratamento.
2. A segunda camada é a *Process Zone* que tem como objetivo transformar os dados, oriundos da camada de Raw Data, para análise utilizando um processamento inicial ou básico. Esse processamento pode ser alguma limpeza, deduplicação, integração, junção, entre outros. Essa camada possibilita os

usuários a ter contato com um dado mais tratado, obtendo uma resposta mais precisa para as dúvidas de negócio.

3. A terceira camada é a *Access Zone* que é a camada aonde os consumidores dos dados já processados têm acesso de forma mais direta a visões consolidadas e consomem os dados tratados, agregados, e se necessário, mascarados de maneira mais direta. Nessa camada pode entrar a governança dos dados com o objetivo de controlar e garantir a segurança, qualidade e consistência do DL.

Os desafios da construção do DL são discutidos por Cuzzocrea (2021) que destaca a importância de se construir uma arquitetura bem pensada, adaptada e eficiente para o modelo de negócio de cada organização. Um outro desafio importante é a necessidade de desenvolver técnicas para que os dados e os metadados em todas as camadas consigam ser reescritos devido a necessidade de manter o dado mais atualizado e relevante ao negócio. Esses desafios também estão ligados a volumetria, como organizar esses dados para que a qualidade esteja alta o suficiente para confiar nos dados? (Harby e Zulkernine 2022).

2.4 Data Lakehouse

O *Data Lakehouse* (DLH) surgiu na necessidade de aprimorar a integração e a eficiência na gestão de dados, combinando as capacidades de armazenamento flexível e escalável do DL com a estruturação e governança de dados típicas dos DW. Essa abordagem híbrida oferece uma solução mais completa para organizações que lidam com grandes volumes de dados de diferentes formatos (estruturados, semiestruturados e não estruturados) e origens, permitindo análises avançadas, aprendizado de máquina e tomada de decisão baseada em dados com maior agilidade e confiabilidade (Čuš e Golec 2023).

As principais características do DLH que Orescanin e Hlupic (2021) citam são:

- Armazenamento de baixo custo de dados estruturados, semiestruturados e não estruturados.

- Utiliza o processamento de *schema on read* e *schema on write* que consiste respectivamente, em uma abordagem onde o esquema dos dados são lidos do armazenamento para a análise e uma abordagem onde o esquema dos dados são definidos no momento de escrita no armazenamento.
- Em relação a agilidade, ele é mais ágil e tem uma configuração mais ajustável quando se comparado com DW e DL.

A seguir uma tabela de comparação entre as três arquitetura de dados (Harby e Zulkernine 2022):

Tabela 1 – Comparação da arquitetura de dados

	<i>Data Warehouse</i>	<i>Data Lake</i>	<i>Data Lakehouse</i>
Tipo de dados	Dados estruturados	Dados semiestruturados e não estruturados	Dados estruturados, semiestruturados e não estruturados
Custo	Caro e consome tempo de ingestão	Barato, rápido e adaptável	Barato, rápido e adaptável
Esquema	Definido	Customizado	Customizado
Qualidade dos dados	Alta confiabilidade	Baixa confiabilidade	Alta confiabilidade
Governança	Rígida, Estruturada e Consistente	Flexível, Descentralizada e Adaptável	Integrada, progressiva e Balanceada

Fonte: Elaborado pela Autora

Um dos principais desafios do DL é a ausência de garantias de transações de atomicidade, consistência, isolamento e disponibilidade (ACID), o que compromete a confiabilidade e a integridade dos dados. Para enfrentar esse problema, surgiram tecnologias no DLH que utilizam formatos de armazenamento de dados populares para análise e incluem camadas de gerenciamento avançadas. As três principais soluções são *Delta Lake*, *Apache Hudi* e *Apache Iceberg* (Tang et al. 2024).

2.5 Delta Lake

O *Delta Lake* é um formato de tabela de código aberto para armazenar dados, aprimorar o armazenamento ao oferecer suporte a transações ACID, realizar otimizações de consulta de alto desempenho, controlar e evoluir esquemas. Ele é composto por dois componentes: arquivos *parquet* e arquivo de *log* com os metadados das transações (“Delta Lake for ETL” 2024).

As principais características do *Delta Lake* é o suporte ACID, o processamento dos escalável dos metadados utilizando o *Apache Spark* e o versionamento dos dados, fornecendo uma fotografia dos dados e permitindo o retorno das versões anteriores (Belov e Nikulchev, 2021).

Segundo Ait Errami et al. (2023), o *Delta Lake* é composto por três componentes principais:

- Camada de Armazenamento *Delta*: Os dados são armazenados no *Delta Lake* e processados pelo *Apache Spark*. Este *design* aproveita a combinação de armazenamento de baixo custo e processamento eficiente, aliado às capacidades analíticas típicas de DW.
- Tabela *Delta*: A tabela organiza os dados em arquivos *parquet* particionados, o que facilita a remoção de dados desnecessários, especialmente em consultas simples, visando otimizar o acesso aos dados relevantes. Outro elemento fundamental da tabela *delta* é o *log* de transações delta, que registra todas as ações realizadas, garantindo a adesão às propriedades das transações ACID.
- Mecanismo *Delta*: Este componente oferece várias otimizações para melhorar o processamento paralelo no *Apache Spark*. Ele utiliza técnicas como a otimização do *layout* de dados com uma abordagem de agrupamento multidimensional. Além disso, o mecanismo delta permite a compactação de arquivos pequenos sem afetar a conformidade ACID.

2.6 Apache Hudi

O *Apache Hudi* é uma plataforma transacional de código aberto que possibilita a criação de tabelas em formato de arquivo aberto como *parquet* e *orc*, oferecendo suporte a operações transacionais como atualização e exclusão dos dados (“What is a Data Lakehouse & How does it Work?” 2024).

Ele foca na otimização da ingestão em tempo real, na identificação e captura de mudanças nos dados para agilizar tanto a ingestão quanto as análises em cenários em que apenas os dados adicionados ao longo do tempo precisam ser acessados. O fluxo incremental melhora o desempenho das consultas ao processar somente os novos dados, evitando a necessidade de reprocessar dados históricos (Ait Errami et al. 2023).

O *Apache Hudi* fornece suporte a operações de *Upsert* e *Delete* através de índices, com conformidade ACID para essas operações como já mencionado anteriormente. Adicionalmente também oferece capacidade de controlar a concorrência de escritas e o rastreamento de linhagem. Essa característica se torna possível por causa da estrutura em diretórios em que o *Apache Hudi* é baseado, onde cada tabela possui um diretório específico com subpastas que representam as partições, dentro dessas partições possuem os dados agrupados em formato *parquet*. Por fim, o *Apache Hudi* também oferece dois tipos de armazenamento de tabela, o *Copy-on-Write* (copiar-ao-escrever) e o *Merge-on-write* (juntar-ao-escrever) (Ait Errami et al. 2023).

O método copiar-ao-escrever ocorre quando existe uma operação de escrita (inserção, atualização ou exclusão) em um conjunto de dados, onde o *Apache Hudi* cria uma nova versão do arquivo apenas com os dados que foram modificados ou inseridos desde a última operação. Isso garante que os dados originais permaneçam imutáveis, mantendo a consistência e permitindo consultas eficientes nas versões anteriores dos dados. Uma das principais vantagens deste método é a consistência dos dados, a fácil reversão para versões anteriores e a eficiência na leitura.

O método juntar-ao-escrever ocorre quando os novos dados são gravados diretamente nos arquivos existentes sem criar versões dos dados, eles são

mesclados com os dados existentes dentro dos arquivos de dados já existentes. A vantagem deste método é uma maior eficiência em relação a armazenamento dos dados, principalmente quando existe uma atualização muito pequena e não necessita criar uma versão inteira novamente. Também reduz a sobrecarga de gerenciamento de muitas versões e facilita o processamento de atualizações de dados, especialmente se o ambiente possuir muitas atualizações.

2.7 Apache Iceberg

O *Apache Iceberg* é um formato de alto desempenho desenvolvido para grandes tabelas analíticas. Ele foi desenvolvido para superar as limitações do *Apache Hive* e resolver problemas de gerenciamento de tabelas em alta escala, suportando dados transacionais através de coleta de metadados. Ele possui um gerenciamento de metadados eficaz que possui rastreamento de versões facilitando a auditoria e recuperação dos dados (Ait Errami et al. 2023).

A arquitetura do *Apache Iceberg* foi projetada baseada em “fotografias”, utilizando uma estrutura em árvore para rastrear os arquivos, realizando conexões entre as fotografias através de uma lista de manifestos, que é como se fosse um índice/sumário de cada fotografia. Cada atualização cria uma fotografia e fica vinculada na lista de manifestos que armazenam os metadados.

O *Apache Iceberg* oferece a confiabilidade e simplicidade das tabelas SQL no contexto de *Big Data*, permitindo que diferentes mecanismos, como *Spark*, *Trino*, *Flink*, *Presto*, *Hive* e *Impala* acessem e manipulem as mesmas tabelas simultaneamente de forma segura.

Como o *Apache Hudi*, o *Apache Iceberg* também utiliza os mesmos métodos de copiar-ao-escrever e juntar-ao-escrever porém com algumas modificações. No método de copiar-ao-escrever, o *Apache Iceberg* utiliza uma estrutura de “fotografias” para garantir a atomicidade, mantendo o controle das versões na lista de manifestos. Já no juntar-ao-escrever, o *Apache Iceberg* foca em garantir a consistência transacional e suportar as operações de atualização de esquemas

2.8 PrestoDB

O *PrestoDB* é um mecanismo de execução de SQL distribuído e de código aberto, desenvolvido pelo Facebook em 2013, e hoje adotado por grandes empresas como Uber, Netflix, Airbnb, Bloomberg e LinkedIn. O *PrestoDB* também está disponível como serviço na nuvem AWS através do Amazon Athena o qual é construído sobre *PrestoDB*. Além disso, ele foi criado para ser adaptável e flexível, oferecendo uma interface SQL ANSI, permitindo consulta em dados armazenados em bancos de dados relacionais, sistemas de processamento streaming e sistemas NoSQL (Sethi et al. 2019).

De acordo com Sethi et al. (2019), o *PrestoDB* é capaz de processar centenas de petabytes de dados e quadrilhões de linhas diariamente em plataformas como o Facebook. Trata-se de um sistema multi-inquilino altamente flexível, projetado para executar simultaneamente centenas de consultas que exigem intensivo uso de memória, I/O e CPU, com a capacidade de escalar para milhares de nós de trabalho enquanto utiliza os recursos do cluster de forma eficiente. O *PrestoDB* foi desenvolvido para garantir alto desempenho, incorporando diversos recursos e otimizações importantes, como a geração de código.

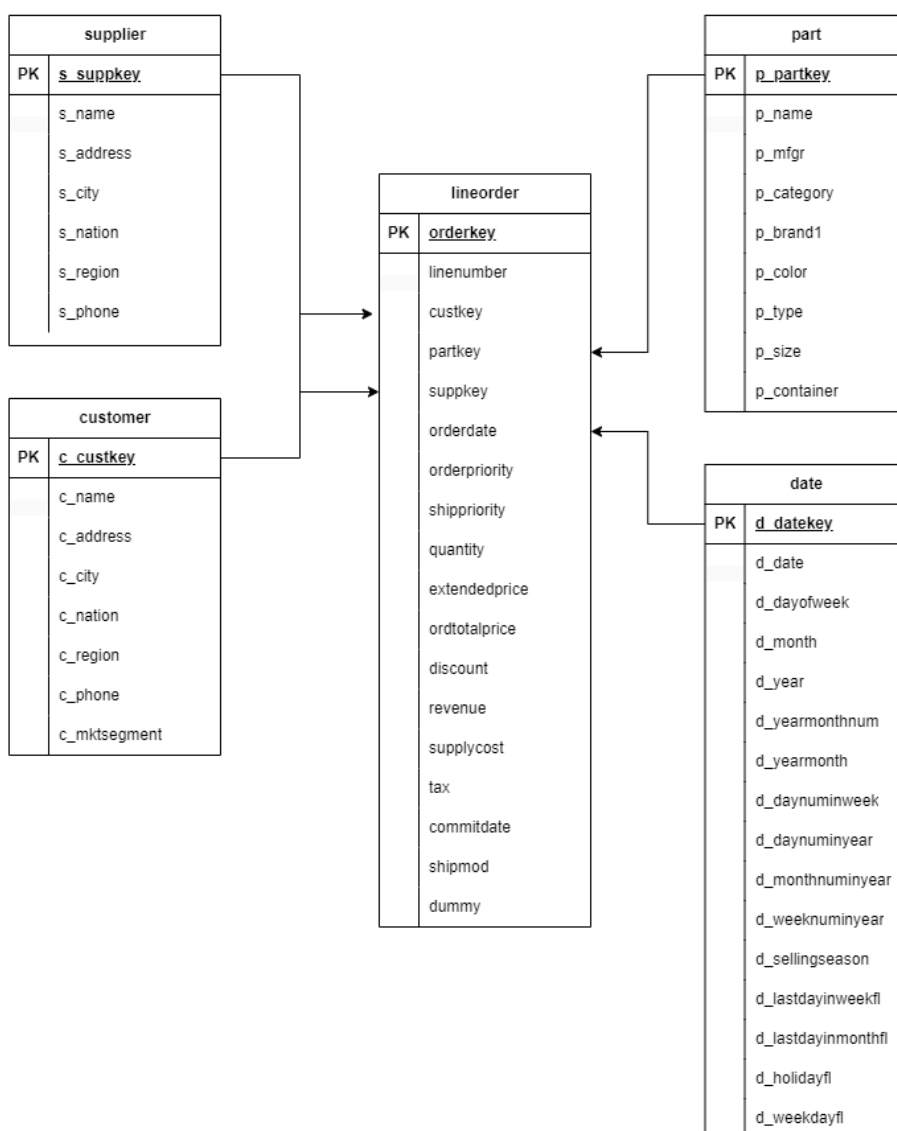
2.9 Star Schema Benchmark

O *Star Schema benchmark* (SSB) é um conjunto de testes criado para analisar o desempenho do sistema de gerenciamento de banco de dados analíticos, principalmente em ambientes que utilizam o modelo *Star Schema* na organização dos dados. O SSB baseia-se no TPC-H que é um outro *benchmark* também com foco no processamento analítico, mas que utiliza o modelo de dados *Snowflake* onde as tabelas são altamente normalizadas e com relações entre as tabelas de dimensão e sub-dimensão. Já o SSB foca em estruturas comuns entre DW onde há uma tabela fato que possui relações diretas com as outras tabelas dimensões, como pode ser visto na Figura 1. O seu uso facilitado em sistemas de *Big Data* e pesquisas vem aumentando e se tornando popular na indústria (Tardío et al. 2022).

Conforme pode ser visto na Figura 1 abaixo, o SSB é organizado por:

- Uma tabela fato que concentra as principais informações das transações como por exemplo valores e quantidades:
 - *Lineorder*: Possui informações de cada linha de pedido, como ID do pedido, quantidade, preço e os Ids das dimensões.
- Quatro tabelas de dimensão que contém detalhes e contexto para os dados, cada dimensão representa uma visão diferente dos dados:
 - *Customer*: Possui dados sobre clientes, como nome, endereço, região, país. Segmentando as vendas pelo perfil de cliente
 - *Supplier*: Possui dados dos fornecedores, como nome, localização e detalhes da empresa
 - *Part*: Possui dados dos produtos, como nome, marca, tipo, categoria. Permitindo classificar os produtos por categoria
 - *Date*: Possui todas as informações sobre as datas, como ano, mês, dia da semana

Figura 1 – Esquema do *Star Schema Benchmark*



Fonte: Elaborado pela Autora (2024)

As tabelas fato e dimensão representam um esquema de dados que é amplamente utilizado para testar e comparar o desempenho de sistemas de banco de dados em tarefas típicas de leitura e atualização, como junção de tabelas, agregações e filtros.

Tendo como o principal objetivo medir o desempenho em operações analíticas medindo o tempo de resposta das consultas, e a eficiência em consultas que contenham agregações e junções. Para medir isso, são disponibilizadas 13 consultas SQL agrupadas em 3 grupos com foco em cada dimensão específica testando junções, filtros e agregações. Essas consultas respondem perguntas que

uma organização teria sobre os dados como por exemplo: Total de vendas por ano, volume de vendas de uma determinada categoria de produto e quais fornecedores venderam mais produtos em períodos específicos.

E por fim, a geração desses dados é através de um gerador de dados com volumetrias diferentes: 1G, 10G, 100G permitindo testar a escalabilidade e desempenho do banco de dados. Após executar as consultas, o tempo de execução é medido e analisado para identificar melhores desempenhos. As 13 consultas disponibilizadas pelo SSB podem ser vistas na seção APÊNDICE A.

2.10 Apache Spark

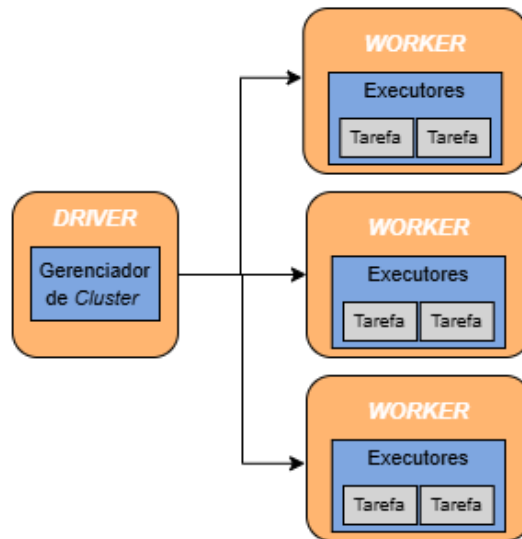
O *Apache Spark* é uma plataforma de execução e processamento de dados em larga escala. Ele oferece suporte a diversas bibliotecas, como SQL e conjunto de dados (*Dataframes*). Diferente de sistemas tradicionais de processamento de dados em *Big Data*, como o *Hadoop*, o *Spark* utiliza armazenamento em memória, o que possibilita um desempenho superior, chegando a ser até 10 vezes mais rápido que o *Hadoop* em tarefas iterativas (Chae e Chung 2019).

Segundo Karthikeya Sahith et al. (2023), o *Spark* proporciona paralelismo implícito de dados, alta tolerância a falhas e facilidade de manutenção por meio de suas Interfaces de Aplicação (APIs). Embora aproveite alguns recursos do *Apache Hadoop*, o *Spark* opera com um *framework* próprio de gerenciamento de *clusters*. Ele oferece interfaces em diversas linguagens, como Python, Java, Scala e R, além de um conjunto robusto de ferramentas e componentes para computação distribuída paralela de alto desempenho em *clusters*. O *Spark* é amplamente adotado no processamento de dados, sendo uma escolha comum entre as organizações.

Sobre a arquitetura, segundo AYDOĞAN e KARCI (2018), o *Spark* adota o modelo *driver-worker*. Conforme a Figura 2 abaixo, o *driver* distribui as tarefas para os nós *workers* e após a execução das tarefas, os resultados são enviados de volta ao *driver*, que finaliza o processamento de dados. Devido a essa arquitetura é possível

escalar o processo aumentando a quantidade de máquinas/*workers* para processar em paralelo.

Figura 2 - Arquitetura de *Spark*



Fonte: Elaborado pela Autora (2024)

3 EXPERIMENTO

O experimento proposto é uma comparação de desempenho de escrita, leitura e atualização dos dados entre os três formatos de armazenamento de dados do DLH mais utilizados: *Delta Lake*, *Apache Hudi* e *Apache Iceberg*. Portanto, as seções seguintes apresentam a arquitetura proposta, o ambiente de desenvolvimento, as tecnologias utilizadas nesse *benchmark* e o resultado do experimento.

3.1 Ambiente de desenvolvimento

O experimento foi desenvolvido utilizando a infraestrutura da AWS, uma plataforma de nuvem que oferece recursos altamente escaláveis e flexíveis para processamento e análise de grandes volumes de dados. A AWS é amplamente utilizada por organizações e pesquisadores devido à sua capacidade de fornecer uma gama abrangente de serviços, que são fundamentais para o armazenamento, processamento e consulta de dados de forma eficiente e econômica. Entre os principais serviços da AWS utilizados no experimento, destacam-se: (1) Amazon S3 (serviço de armazenamento de arquivos de forma distribuída), que foi responsável pelo armazenamento dos dados; (2) Amazon Athena, o qual permitiu a execução de consultas SQL diretamente sobre os dados armazenados no S3, sem a necessidade de provisionar servidores adicionais, o que reduz significativamente o custo e o tempo de manutenção; (3) *Glue Data Catalog*, o qual é responsável por armazenar os metadados de forma centralizada e integrada com outros serviços da AWS, facilitando o gerenciamento e acesso aos dados em diferentes fontes e formatos; e (4) Amazon EMR, o qual foi utilizado para criar *clusters* de processamento capazes de lidar com grandes cargas de trabalho. O EMR ele tem como objetivo o processamento de dados em larga escala utilizando *frameworks* como *Apache Hadoop*, *Apache Spark*, *Apache Hive*, *PrestoDB* entre outros. Com o EMR, pode ser facilmente criado *clusters* configuráveis para processar uma volumetria grande de dados, seja para análises em *batch* ou em tempo real, sem precisar se preocupar com a infraestrutura.

A integração entre EMR, S3 e Athena permite que os dados sejam processados de forma eficiente e as consultas sejam realizadas de maneira rápida, aproveitando as características de escalabilidade e flexibilidade da AWS. Além disso, o EMR oferece suporte a *frameworks* mais atuais como *Apache Hudi*, *Apache Iceberg* e *Delta Lake*. Dessa forma, resulta em um suporte avançado para gerenciamento de dados transacionais e operações como leitura, atualização e inserção de dados, otimizando o desempenho conforme o volume de dados cresce.

Também foi utilizado as treze consultas sugeridas pelo SSB para gerar as métricas de leitura das três tecnologias. Essas treze consultas possuem diversas operações como junção de tabelas, contagem e soma de campos, filtros, agrupamento de dados e ordenação para medir de forma mais completa o desempenho de cada tecnologia.

Para criar o ambiente de processamento dos dados, foi utilizado um *cluster* Amazon EMR de sistema operacional Amazon Linux com instâncias de tipo *m4.large*. Esse *cluster* foi configurado com um nó primário, um nó núcleo e dois nós de tarefa, cada um equipado com 2 vCPUs, 8 GB de memória e 64 GB de armazenamento EBS (*Elastic Block Store*). A configuração com múltiplos nós e uma arquitetura escalável de armazenamento e processamento permitiu o desempenho necessário para rodar as consultas sobre o conjunto de dados de forma eficiente.

Para gerar os dados necessários para o experimento, foi seguido os seguintes passos: (1) Utilizou-se o repositório ¹ que contém um gerador de dados especialmente projetado para *benchmarks*, o SSB. Esse gerador de dados é amplamente utilizado em testes de desempenho de sistemas de banco de dados, simulando operações de leitura, escrita e atualização de tabelas estruturadas em *Star Schema*.

¹ <https://github.com/electrum/ssb-dbgen>

O repositório foi clonado diretamente na máquina Linux no ambiente de desenvolvimento configurado na AWS, e o código fonte foi compilado utilizando o comando *make*, que é responsável por compilar e gerar o binário que executa a geração dos dados conforme os parâmetros definidos no *script*, garantindo que o processo de criação dos dados fosse realizado sem falhas. (2) Em seguida, foram configurados dois cenários de volumetria para a comparação de desempenho: um com fator de escala 1, correspondente a 1 GB de dados, e outro com fator de escala 10, resultando em 10 GB de dados. Para criar o primeiro conjunto de dados, foi utilizado o comando “*dbgen -s 1 -T a*”, onde o parâmetro “-s 1” define o tamanho do conjunto de dados, nesse caso 1G e o “-T a” significa que todas as cinco tabelas devem ser geradas. E, para o segundo, o comando “*dbgen -s 10 -T a*”. Esses diferentes tamanhos de conjuntos permitiram testar desempenho em relação ao aumento de volumetria. Após a execução do gerador, os arquivos de dados no formato “.tbl” são enviados para o Amazon S3 utilizando o comando “*aws s3 cp*” do Linux.

Além dos arquivos iniciais, uma segunda série de arquivos foi gerada contendo os dados atualizados da tabela fato *lineorder*, através do comando “*dbgen -s 1 -r 20 -U 4*” que indica a escala de 1G no parâmetro -s 1, a taxa de atualização dos dados de 0,20% e -U 4 que define 4 arquivos inserção e exclusão. Esses arquivos serão utilizados no processo de atualização dos dados, para simular um ambiente transacional de uma arquitetura de dados, o SSB já possui essa geração de arquivos com o mesmo esquema, porém com dados atualizados.

Foi utilizado o *PrestoDB* para realizar as consultas, aproveitando a flexibilidade de execução dentro do *cluster* EMR, o que permitiu um melhor controle das instâncias e da capacidade computacional utilizada. Diferentemente do Athena, onde a infraestrutura é gerenciada automaticamente pela AWS e não permite visibilidade sobre os nós de execução, o uso do *PrestoDB* em um *cluster* EMR proporciona a possibilidade de ajustar o número de máquinas e tipos de instâncias. Dessa forma, garantia uma execução mais transparente e customizável das consultas SQL.

As configurações de cluster para cada tecnologia foram diferentes, pois cada uma possui requisitos específicos de infraestrutura para otimizar o desempenho. Esses

ajustes levam em consideração características como a instalação de bibliotecas necessárias, a forma como gerenciam metadados, e o suporte para operações de leitura e escrita:

- *Delta Lake*: O *cluster* criado tem a versão emr-7.0.0 do Amazon EMR com o sistema operacional Amazon Linux, com um conjunto de aplicações que inclui *Spark 3.5.0*, *Hadoop 3.3.6*, *Hive 3.1.3*, e *JupyterHub 1.5.0*, além de outras ferramentas como *Hue 4.11.0* e *PrestoDB 0.283*. Esse conjunto de aplicativos fornece uma estrutura confiável para processar e gerenciar os dados de maneira eficiente, permitindo fazer análises rápidas com o Spark e consultas SQL distribuídas com o *PrestoDB*. As máquinas utilizadas no cluster são instâncias do tipo *m4.large*, que oferecem 2 vCPUs e 8 GB de memória e cada instância tem um armazenamento EBS de 64 GB, distribuído em dois volumes. Essas máquinas estão configuradas como "Sob Demanda", o que significa que serão cobradas de acordo com o tempo de uso, oferecendo flexibilidade para ajustar a capacidade de processamento. As configurações específicas para habilitar o *Delta Lake* no *cluster* foram definidas através de um arquivo JSON, incluindo a configuração de *software "delta.enabled": "true"* dentro da classificação "*delta-defaults*". Permitindo que ele seja instalado no *cluster* e habilitado a utilização de funcionalidades avançadas como controle de versão de dados e transações ACID.
- *Apache Hudi*: A partir da versão 5.28.0 do EMR, o *Apache Hudi* já vem instalado por padrão quando o *Spark* é instalado. Com isso, foi necessário habilitar um *cluster* com as mesmas configurações de máquina do *Delta*, porém com o conjunto de aplicativos diferentes que seriam os: *Hadoop 3.3.6*, *Spark 3.5.0*, *Hive 3.1.3*, *PrestoDB 0.283*, *Jupyter Enterprise Gateway 2.6.0*, *Hue 4.11.0*, *Livy 0.7.1* e o *Tez 0.10.2*. Não foi necessário configurações específicas utilizando um arquivo JSON.
- *Apache Iceberg*: Para a configuração do *Apache Iceberg*, foi adicionado um arquivo de *bootstrap* salvo no *bucket* S3 e chamado na criação do *cluster* com o conteúdo "*connector.name=iceberg*" e "*iceberg.catalog.type=glue*" que tem como objetivo respectivamente, definir o conector com as tabelas no cluster e especificar que o catálogo de dados deve ser o serviço *Glue Catalog*. As

configurações do *cluster* foram as mesmas do *Delta Lake* e do *Apache Hudi*, com a exceção de adicionar uma configuração a mais nas configurações de software do *cluster* para que ele ative o *Apache Iceberg* definindo a propriedade “*iceberg.enabled*” como “*true*”. Em seguida, foi incluído o caminho do JAR necessário para o funcionamento do *Apache Iceberg* no *Spark*, usando a configuração “*spark.jars*” com o caminho “*/usr/share/aws/iceberg/lib/iceberg-spark3-runtime.jar*”.

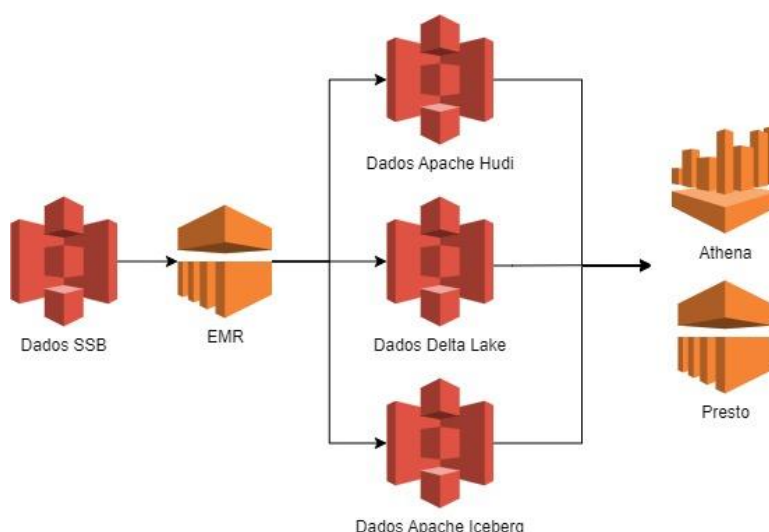
Para integrar o *Apache Iceberg* com o catálogo do *Spark*, adicionou-se “*spark.sql.catalog.spark_catalog*” como “*org.apache.iceberg.spark.SparkCatalog*”, especificando o tipo de catálogo com “*spark.sql.catalog.spark_catalog.type*” configurado como “*hadoop*”. Definiu-se também o diretório no S3 onde os dados e metadados do *Apache Iceberg* serão armazenados, utilizando “*spark.sql.catalog.spark_catalog.warehouse*” com o caminho “*s3://iceberg-pece2024/iceberg-metadata/db/iceberg_table/data*”. Por fim, a extensão necessária para o *Apache Iceberg* no *Spark*, configurando “*spark.sql.extensions*” como “*org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions*”. Todas essas configurações foram salvas assim que o *cluster* foi criado, deixando um ambiente preparado para realizar as ingestões dos dados com o *Apache Iceberg*.

3.2 Arquitetura Proposta

A arquitetura proposta para o *benchmark* adota o conceito de DLH, combinando a flexibilidade de um DL com as capacidades de um DW, oferecendo uma estrutura unificada para o processamento de dados e consultas analíticas. Na Figura 3 apresentamos a arquitetura proposta, a qual é composta por duas camadas de dados: a primeira, com os dados gerados via SSB, a qual serve como base para a análise; e a segunda, com os dados segregados por tecnologia, aproveita as vantagens específicas de cada uma, como atualizações rápidas e incrementais no *Apache Hudi*, consultas otimizadas e garantias ACID no *Delta Lake*, e eficiência de leitura em grande escala no *Apache Iceberg*. Essa estrutura possibilita a comparação de métricas de desempenho, como tempo de ingestão, tempo de leitura

e eficiência de atualização, fornecendo uma visão detalhada sobre o desempenho das diferentes soluções de armazenamento. A migração dos dados entre os *buckets* é realizada pelo EMR, que utiliza uma instância Amazon Linux para executar *scripts* em *PySpark*. Os dados são então transferidos para três *buckets* distintos: Dados *Apache Hudi*, Dados *Delta Lake* e Dados *Apache Iceberg*, cada um com os arquivos e processos específicos da tecnologia correspondente. Por fim, o serviço Athena e outro EMR são usados para executar o *PrestoDB*, realizar consultas e validar as métricas de desempenho.

Figura 3 – Arquitetura Proposta



Fonte: Elaborado pela Autora (2024)

3.3 Detalhamento do Experimento

O experimento proposto visou realizar uma comparação detalhada do desempenho de três tecnologias populares de armazenamento de dados utilizadas em arquiteturas DLH: *Delta Lake*, *Apache Hudi* e *Apache Iceberg*. Essas tecnologias são amplamente adotadas para operações de escrita, leitura e atualização de dados, sendo fundamentais para o processamento de grandes volumes de informações em

ambientes de dados modernos. A análise focou em medir e comparar a eficiência de cada uma dessas tecnologias em diferentes operações e volumes de dados.

Para as operações de escrita e atualização, foram utilizados *scripts* em *PySpark* que realizaram as operações nas tabelas configuradas em *Apache Hudi*, *Apache Iceberg* e *Delta Lake*. Cada operação foi executada três vezes para cada tecnologia a fim de reduzir as variações nos tempos de execução e garantir uma média mais acurada, permitindo uma análise mais confiável do desempenho de cada tecnologia. Além disso, as operações foram realizadas para dois volumes distintos de dados: 1 GB e 10 GB, simulando diferentes cenários de uso e avaliando como cada sistema de armazenamento lida com menores e maiores volumes de dados. A repetição das operações e a utilização de diferentes volumes foram fundamentais para obter uma visão detalhada do desempenho das tecnologias em situações variadas, além de ajudar a identificar possíveis gargalos ou otimizações necessárias em cada sistema.

Nas operações de leitura, foi utilizado o *PrestoDB*, um mecanismo de consulta distribuída, executado no *cluster* Amazon EMR para realizar consultas nos dados tanto após a operação de escrita quanto após a atualização, permitindo observar o impacto de cada operação nas consultas subsequentes. Foram executadas 13 consultas distintas definidas pela metodologia SSB que possuem junções e filtros, e uma consulta simples nos dados via Athena. Essas execuções ocorreram três vezes para cada volumetria e operação, e a média dos tempos de execução foi registrada. Com essa abordagem, foi possível avaliar o impacto dos diferentes volumes de dados no desempenho de leitura de cada tecnologia, identificando qual oferece o melhor desempenho em operações de leitura e como cada uma se comporta ao lidar com maior carga de dados.

Durante o experimento, foi possível executar o *PrestoDB* para realizar as consultas de leitura em *Apache Hudi* e *Delta Lake* com êxito. No entanto, ao tentar executar as consultas no *Apache Iceberg*, ocorreu um erro que impediu a execução via *PrestoDB*: “*failed: Not a Hive table 'default.iceberg_table'*”. Esse erro ocorreu porque o *Apache Iceberg*, ao contrário de *Apache Hudi* e *Delta Lake*, utiliza um formato de tabela que não é compatível com o *Hive* diretamente, o que gerou a falha ao tentar executar as consultas. Apesar de diversas tentativas de resolver essa

incompatibilidade, incluindo ajustes nas configurações do catálogo e metadados, não foi possível fazer o *PrestoDB* funcionar corretamente com o formato de tabela do *Apache Iceberg*. Por isso, as consultas não puderam ser executadas como originalmente planejado e para contornar essa limitação, todas as métricas do *Apache Iceberg* foram coletadas utilizando o Athena e para garantir a consistência na comparação entre as tecnologias, as consultas de leitura no *Apache Hudi* e no *Delta Lake* também foram executadas através do Athena. Dessa forma, foi possível realizar uma avaliação comparativa equilibrada entre os três sistemas de armazenamento de dados.

Durante as operações de atualização de dados, foi executado um *script* que consumia os dados armazenados nos *buckets* correspondentes às tabelas de cada tecnologia (*Apache Hudi*, *Apache Iceberg* e *Delta Lake*), além dos dados de atualização gerados via SSB. O *script* identificava os registros que precisavam ser atualizados, utilizando as chaves de atualização baseadas nos campos *orderkey* e *linenumber*. Com isso, ele realizava as atualizações e inserções de dados, aplicando o mecanismo específico de cada tecnologia para garantir que os dados novos fossem integrados corretamente e que as atualizações fossem realizadas de acordo com as características de cada sistema de armazenamento.

O primeiro *bucket* no S3 contém os dados no formato “.tbl” gerados via SSB das tabelas: *lineorder*, *part*, *supplier*, *date* e *customer*. Utilizando o *cluster* EMR, os dados são ingeridos para três *buckets* diferentes: *Apache Hudi*, *Delta Lake* e *Apache Iceberg*. Cada sistema tem seu próprio *script* de ingestão e para que o sistema execute de acordo com o esperado, foi preciso configurar o *cluster* de maneiras diferente, a próxima seção detalhará as configurações e diferenças de inicialização das três tecnologias.

3.3.1 Utilização do Delta Lake

Para utilização do *Delta Lake* com *Spark* foi necessário iniciar uma sessão do *Spark* configurada e importar a biblioteca *pyspark.sql*. Primeiro passo foi conectar no *bucket* “Dados SSB” e recuperar o arquivo de uma tabela utilizando o formato CSV

com o delimitador "|". Esse arquivo foi lido em um *DataFrame Pyspark*, e em seguida, o *script* configurou o caminho para uma tabela *Delta* armazenada no S3. O objetivo era gravar os dados do *DataFrame* na tabela *Delta* utilizando o modo "append", permitindo que novos registros sejam adicionados sem substituir os existentes.

Para monitorar o desempenho, o código mediu o tempo de execução da operação de escrita na tabela *Delta*. Após a escrita do dado, o código calculou o tempo total em segundos e o converteu para minutos, imprimindo o resultado. Além disso, um *DataFrame Pyspark* chamado *config_df* foi criado para armazenar informações sobre a configuração e execução da operação, como a tecnologia utilizada (*Delta Lake*), tempo de execução, operação realizada (Escrita ou Atualização), nome da tabela e observações adicionais. A Figura 4 mostra os exemplos dos registros salvos na tabela *config_df*.

Figura 4 – Resultado da tabela *config_df*

tech	execution_time	operation ▼	table_name ▼	start_time ▼	end_time ▼	observation
Delta Lake	0.7	insert	delta_table	2024-08-10 17:55:05	2024-08-10 17:55:47	terceira execução
Delta Lake	0.76	insert	delta_table	2024-08-10 17:52:42	2024-08-10 17:53:27	segunda execução
Delta Lake	0.83	insert	delta_table	2024-08-10 17:49:51	2024-08-10 17:50:41	primeira execução

Fonte: Elaborado pela Autora (2024)

3.3.2 Utilização do Apache Hudi

Para a ingestão no *Apache Hudi* foi configurada uma sessão do *Spark* e importado a biblioteca *pyspark.sql*. Em seguida, leu o arquivo de entrada de uma tabela localizado no *bucket* "Dados SSB", usando o delimitador "|" para depois carregar os dados em um *DataFrame Pyspark*, foi adicionado no processo opções específicas do *Apache Hudi*, como o tipo de tabela *Copy-on-Write* (COW), onde cada atualização substitui o arquivo inteiro, e habilita a sincronização com o *Hive*, permitindo que a tabela seja facilmente consultada em catálogos de metadados.

Após a definição das configurações no *script* foi medido o tempo necessário para realizar a operação de inserção e era salvo o *DataFrame* em uma tabela *Apache*

Hudi no Amazon S3. O tempo de execução foi calculado e convertido para minutos, sendo exibido ao final e alimentando a mesma tabela *config_df* que possui todos os tempos salvos do processamento de inserção.

3.3.3 Utilização do Apache Iceberg

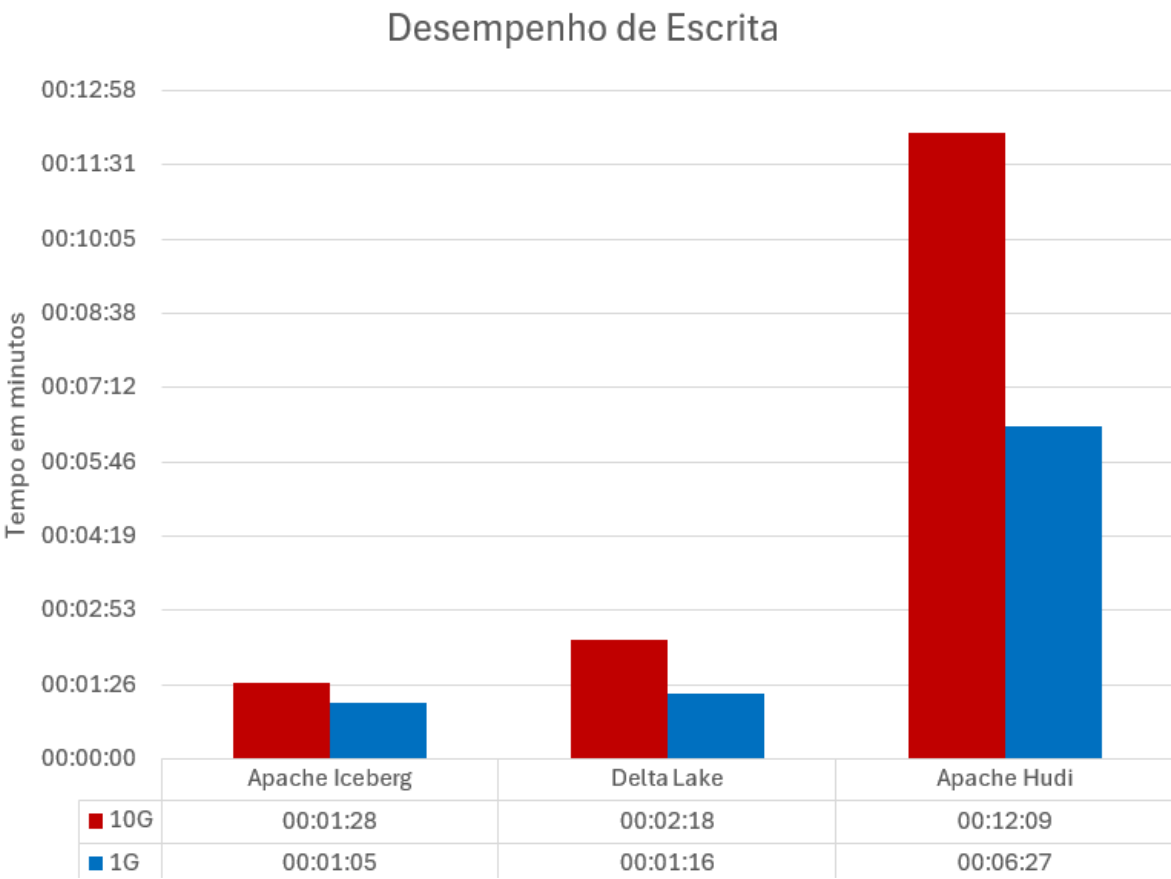
Para a ingestão dos dados no *Apache Iceberg*, foi configurado uma sessão *Spark* adicionando as extensões e o catálogo para armazenar dados em um sistema de arquivos *Hadoop*. O *script* era utilizado para leitura do arquivo de entrada localizado no *bucket* “Dados SSB”, utilizando o delimitador “|”. Os dados desse arquivo foram carregados em um *DataFrame*. Além disso também foi configurado algumas especificações para executar o *Apache Iceberg*, como por exemplo o tipo da tabela *Copy-on-Write* (COW), onde cada atualização no processo de escrita é reescrito para se ter uma consistência nos dados e uma leitura mais rápida.

Em seguida, era utilizado *script* para iniciar a contagem do tempo e realizar a operação de inserção dos dados armazenando esses dados no caminho especificado no S3. Após a gravação dos dados, também era armazenado o tempo da operação na mesma tabela que *Delta Lake* e *Apache Hudi* e *config_df*.

4 RESULTADOS

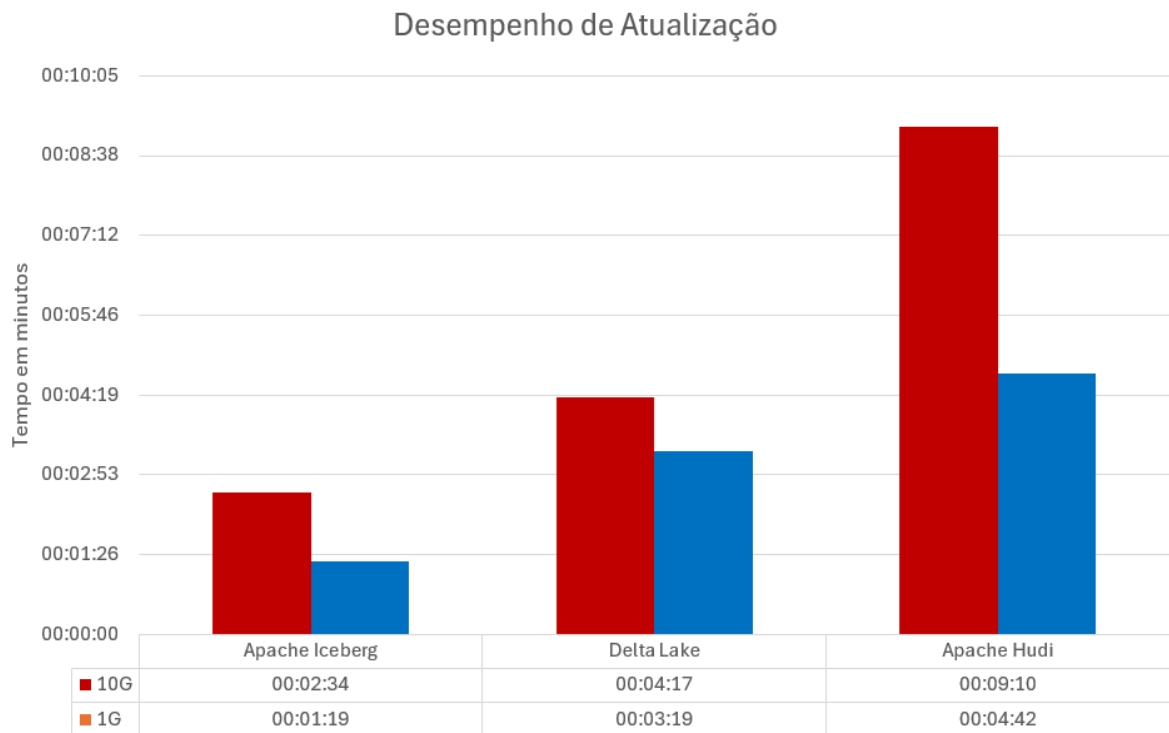
Os resultados do experimento realizado serão apresentados e discutidos nesta seção. Para obtenção dos resultados foi comparado o desempenho das tecnologias *Apache Hudi*, *Delta Lake* e *Apache Iceberg* em operações de escrita e atualização de dados, considerando volumes de 1GB e 10GB. As Figuras 5 e 6 ilustram as diferenças no tempo de execução para essas operações, onde respectivamente mostram o desempenho de escrita e atualização dos dados.

Figura 5 – Resultado de desempenho de Escrita



Fonte: Elaborado pela Autora (2024)

Figura 6 – Resultado de desempenho de Atualização



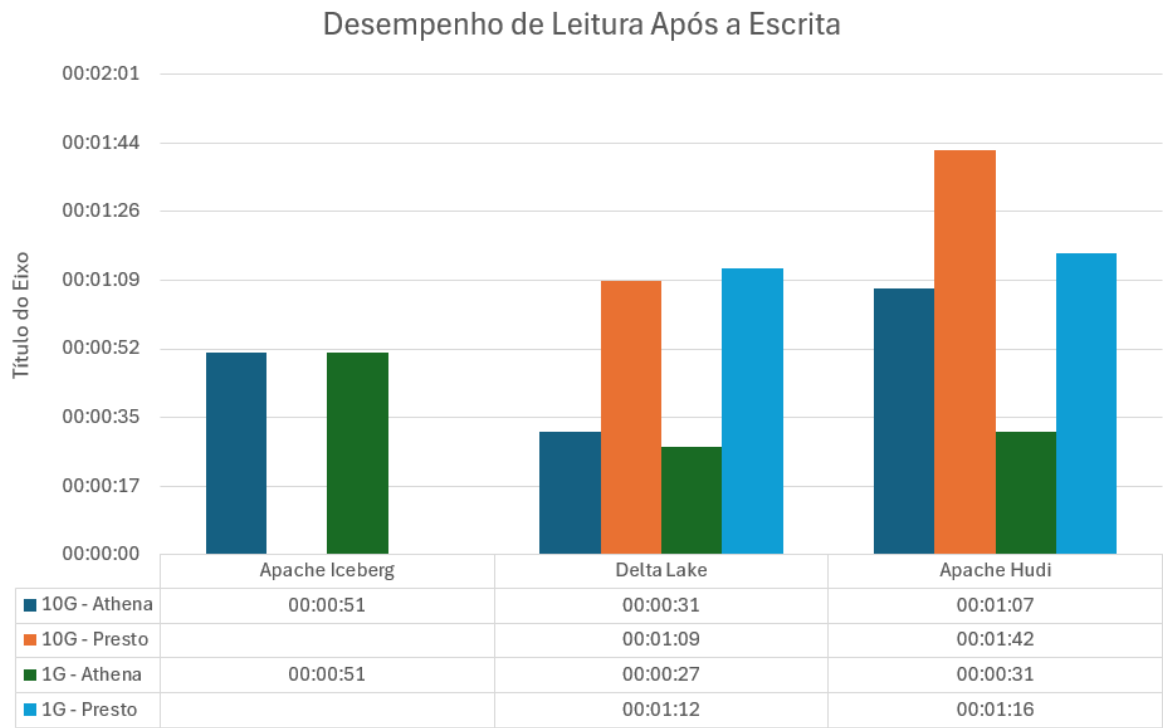
Fonte: Elaborado pela Autora (2024)

Os resultados mostram que na operação de escrita, o *Apache Iceberg* foi mais eficiente. Nota-se que o tempo do *Apache Iceberg* para a operação de escrita ao passar de 1GB para 10GB de dados, aumentou em 23 segundos (35%) enquanto para o *Delta Lake* esse aumento foi de 62 segundos (81%) e para o *Apache Hudi* foi de 342 segundos (88%).

Na atualização dos dados, o *Apache Iceberg* mais uma vez registrou o menor tempo de operação quando comparado com as outras tecnologias, mas pode-se observar que ao ser comparado os conjuntos de 1GB e 10GB de cada tecnologia, o *Delta Lake* obteve um aumento de tempo de 58 segundos (29,14%), enquanto para o *Apache Iceberg* esse aumento foi de 75 segundos (94,94%) e o *Apache Hudi* foi de 268 segundos (95,21%), indicando que o *Delta Lake* foi menos afetado pela variação de volumetria.

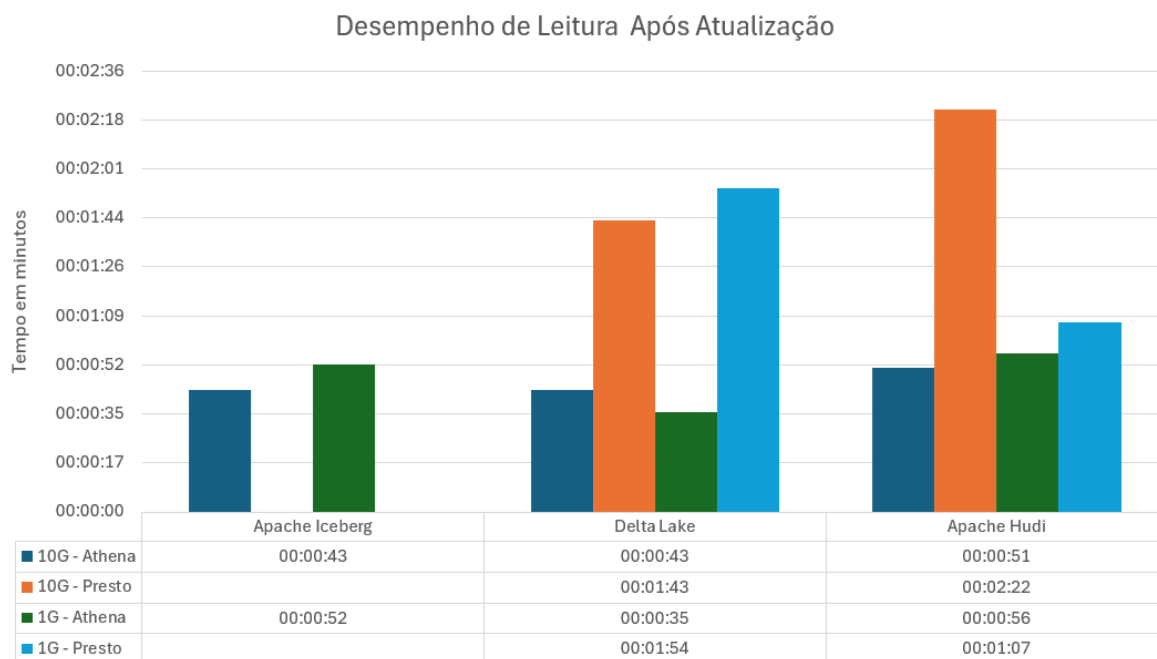
Em relação as operações de leitura, foram analisados dois momentos de leitura diferentes, um após a escrita dos dados e outro após a atualização dos dados, os resultados obtidos são mostrados nas Figuras 7 e 8.

Figura 7 – Resultado de desempenho de Leitura após Escrita



Fonte: Elaborado pela Autora (2024)

Figura 8 – Resultado de desempenho de Leitura após Atualização



Fonte: Elaborado pela Autora (2024)

Em relação as operações de leitura, como mencionado na seção 3.3, não foi possível executar as consultas via *PrestoDB* para o *Apache Iceberg* e por isso as comparações foram feitas também via *Athena*. Aprofundando melhor na operação de leitura após escrita, pode-se observar que para o motor de execução *PrestoDB*, o *Delta Lake* demonstrou ser o mais rápido ao ler os dados quando comparado com o *Apache Hudi*. E quando comparado as leituras dos dados de 1G e 10G, percebe-se que o *Delta Lake* obteve um decréscimo de 3 segundos (4,1%) no tempo de leitura enquanto o *Apache Hudi* mostrou um acréscimo no tempo de 26 segundos (34,2%).

Ainda sobre a leitura dos dados após a escrita, mas agora via *Athena* foi constatado que tanto o *Delta Lake* quanto o *Apache Hudi* tiveram um acréscimo de 3 segundos (14,8%) e 36 segundos (116,3%) respectivamente, quando comparados a volumetria de 1G e 10G, o *Apache Iceberg* permaneceu com o mesmo tempo de execução da consulta de 51 segundos.

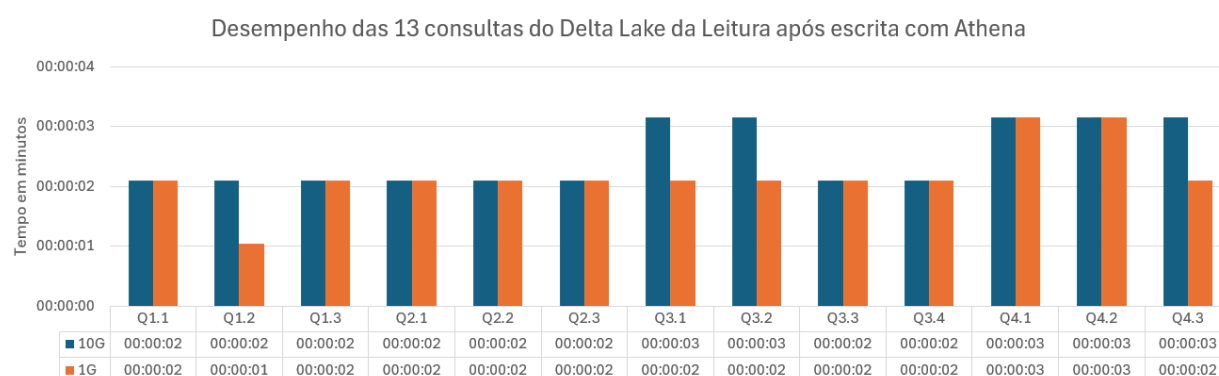
Sobre a operação de leitura após a atualização utilizou-se as treze consultas propostas pelo SSB Cada consulta foi executada três vezes para que o resultado fosse o mais consistente possível. Então, visualizando primeiramente a leitura via *PrestoDB*, o *Apache Hudi* com 1G de dados obteve a leitura mais rápida em 70,1% quando comparado ao *Delta Lake*, porém ao comparar com a volumetria de 10G o

cenário se inverte e o *Delta Lake* obteve o desempenho mais rápido em 37,8%. Na sequência, comparando a massa de dados dentro da própria tecnologia o *Delta Lake* obteve um decréscimo de 9 segundos (9,6%) no tempo ao ler uma volumetria maior, já o *Apache Hudi* obteve um aumento de 75 segundos (111,9%) na leitura dos dados de 10G.

Via Athena o cenário obtido foi diferente, tanto o *Apache Iceberg* quanto o *Apache Hudi* tiveram decréscimos na leitura dos dados de 10G quando comparado com 1G de 9 segundos (17,3%) e 5 segundos (8,9%) respectivamente. Enquanto o *Delta Lake* possuiu um aumento de 7 segundos representando 22,8%.

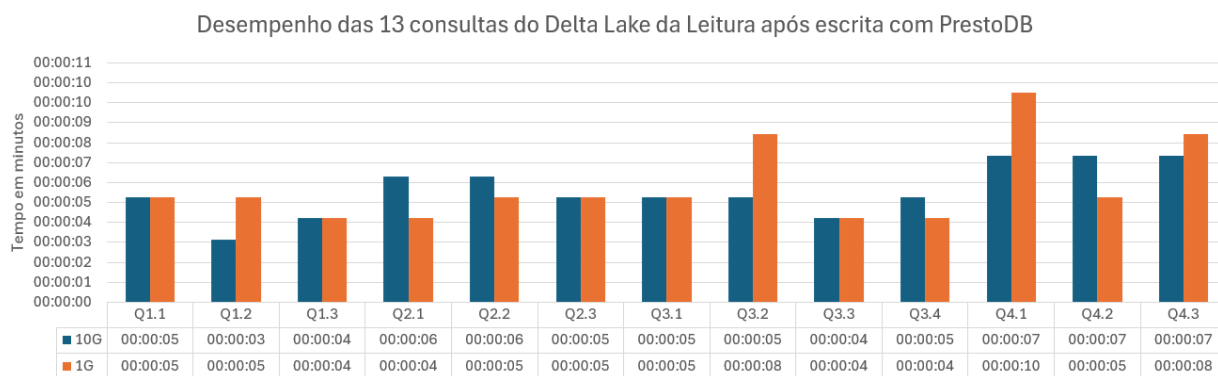
Adentrando nas treze consultas realizadas, as figuras 9 a 13 mostram o desempenho de cada tecnologia para cada consulta na leitura dos dados via Athena e *PrestoDB* após a escrita.

Figura 9 – Resultado do desempenho das 13 consultas do Delta Lake da Leitura após a escrita lidos com o Athena



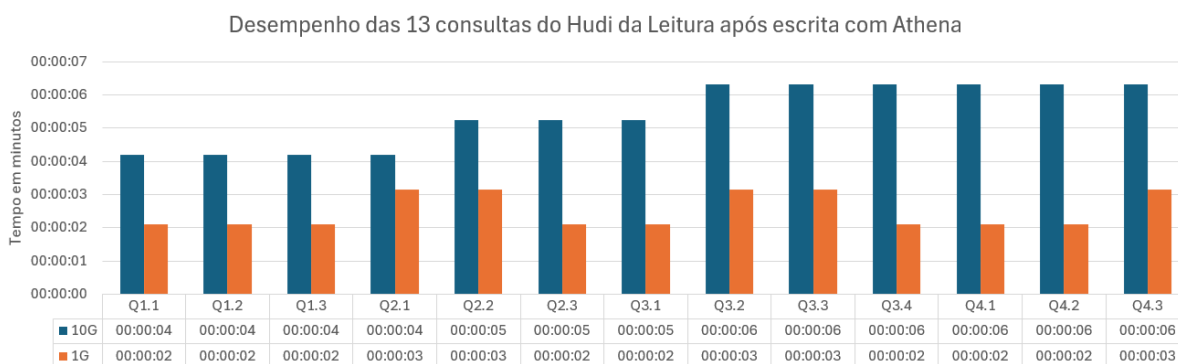
Fonte: Elaborado pela Autora (2024)

Figura 10 – Resultado do desempenho das 13 consultas do Delta Lake da Leitura após a escrita lidos com o PrestoDB



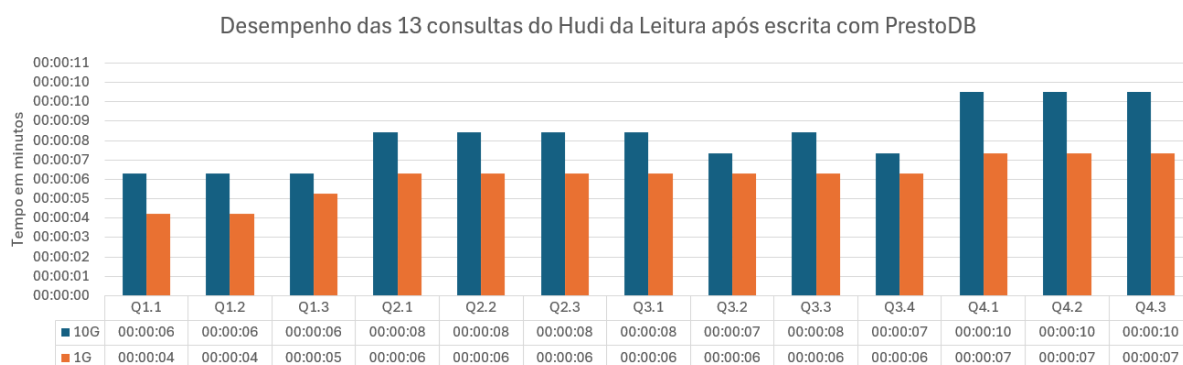
Fonte: Elaborado pela Autora (2024)

Figura 11 – Resultado do desempenho das 13 consultas do Apache Hudi da Leitura após a escrita lidos com o Athena



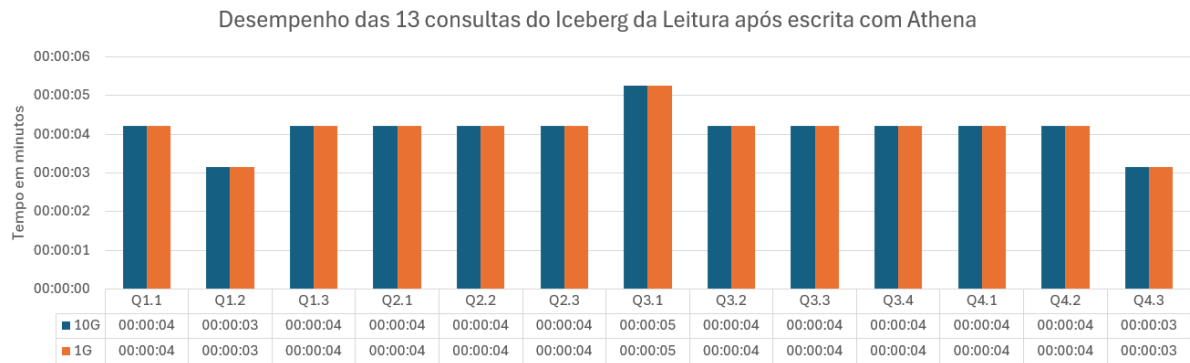
Fonte: Elaborado pela Autora (2024)

Figura 12 – Resultado do desempenho das 13 consultas do Apache Hudi da Leitura após a escrita lidos com o PrestoDB



Fonte: Elaborado pela Autora (2024)

Figura 13 - Resultado de desempenho das 13 consultas do Apache Iceberg da Leitura após a escrita lidos com o Athena



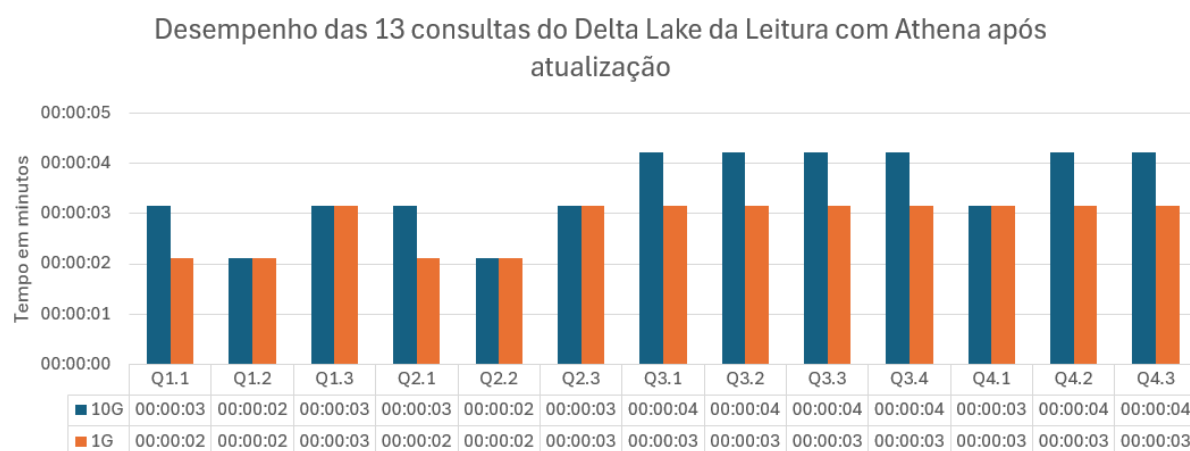
Fonte: Elaborado pela Autora (2024)

A partir dos resultados mostrados da Figura 9 a Figura 13, foi possível analisar o desempenho em relação à complexidade das consultas. Como observado, o *Delta Lake* tende a apresentar tempos de leitura maiores em consultas mais complexas (Q3.2, Q4.1, Q4.2 e Q4.3), com mais de 3 segundos no Athena, e com mais de 7 segundos no *PrestoDB*.

Já o *Apache Hudi* se destacou no crescente tempo na leitura dos dados de 10G via Athena, conforme a complexidade das consultas aumentam. A partir da consulta Q3.2 o período se estabelece e mantém. Via *PrestoDB* também com os dados de 10G, o *Apache Hudi* apresentou um aumento de 25% no tempo das últimas consultas (Q4.1, Q4.2 e Q4.3) quando comparado com a consulta Q3.3 que executou em 8 segundos. Em contrapartida, o *Apache Iceberg* via Athena se destacou por manter o tempo de execução para ambas as volumetrias.

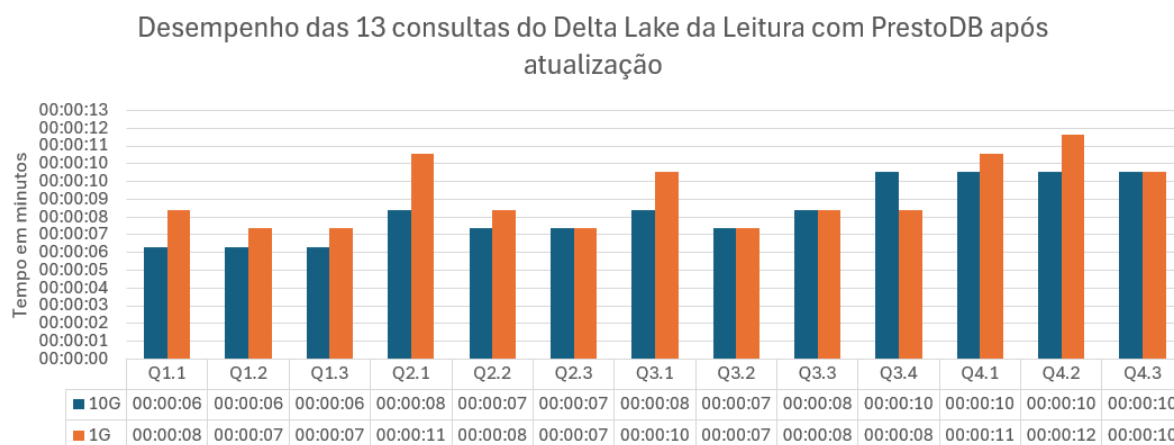
Em relação ao desempenho de cada tecnologia para cada consulta na leitura dos dados via Athena e *PrestoDB* após a atualização, os resultados são mostrados da Figura 14 a Figura 18.

Figura 14 – Resultado do desempenho das 13 consultas do Delta Lake da Leitura após a atualização lidos com o Athena



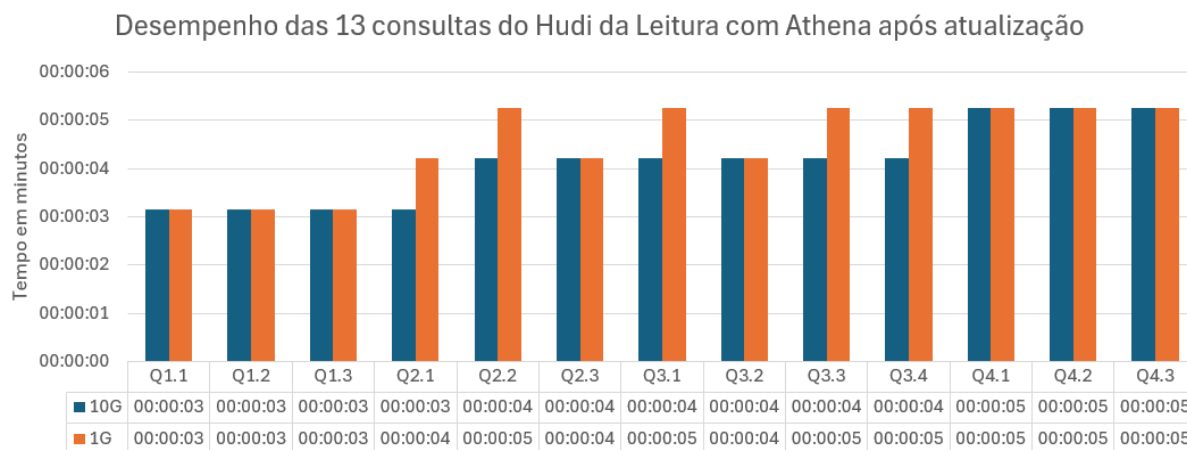
Fonte: Elaborado pela Autora (2024)

Figura 15 – Resultado do desempenho das 13 consultas do Delta Lake da Leitura após a atualização lidos com o PrestoDB



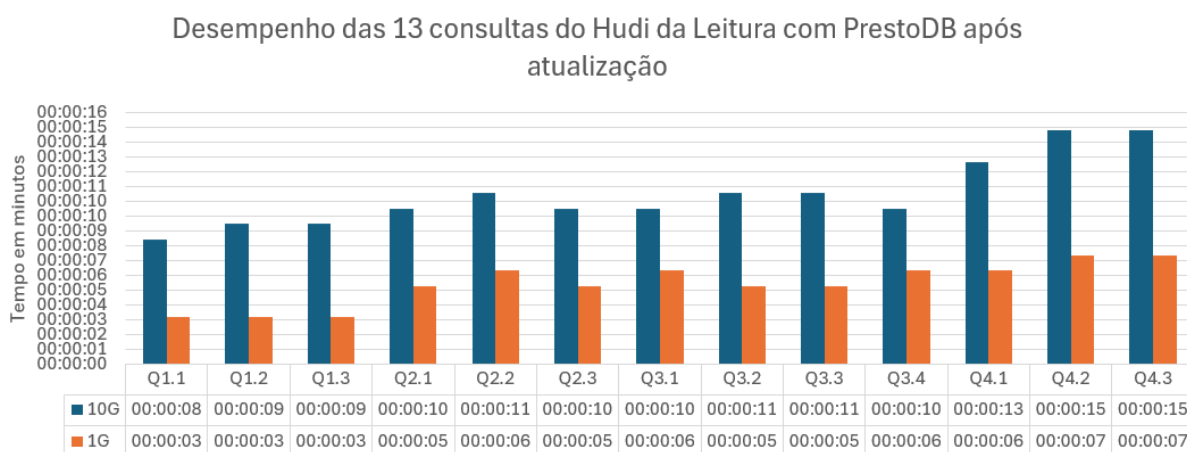
Fonte: Elaborado pela Autora (2024)

Figura 16 – Resultado do desempenho das 13 consultas do Hudi da Leitura após a atualização lidos com o Athena



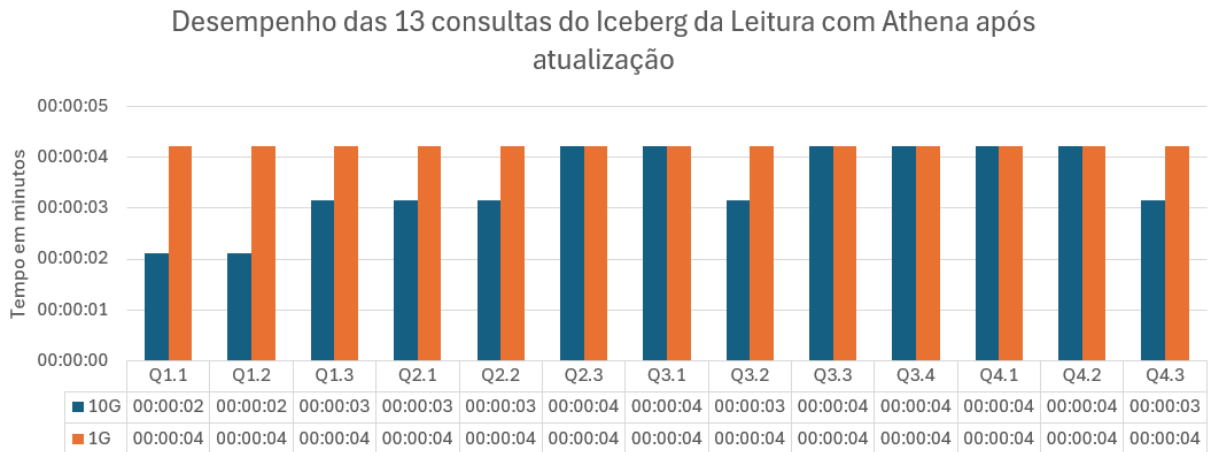
Fonte: Elaborado pela Autora (2024)

Figura 17 – Resultado do desempenho das 13 consultas do Hudi da Leitura após a atualização lidos com o PrestoDB



Fonte: Elaborado pela Autora (2024)

Figura 18 – Resultado do desempenho das 13 consultas do Apache Iceberg da Leitura após a atualização lidos com o Athena



Fonte: Elaborado pela Autora (2024)

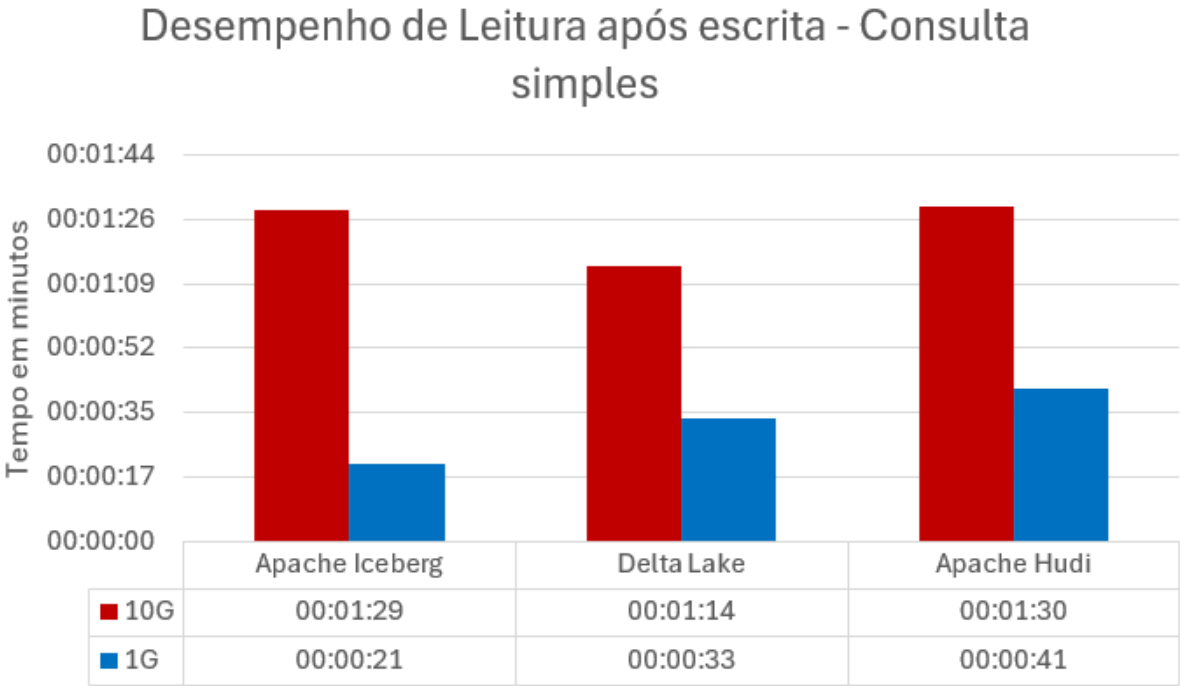
Pode ser observado que as consultas após a atualização dos dados, seguem o padrão de aumento do tempo pela complexidade da leitura após a escrita *no Delta Lake*. Algumas consultas executadas via *PrestoDB* obtiveram o aumento de 100% do tempo de execução, como foi o caso da Q3.3 porém existem outras consultas que aumentaram 1 segundo (20%) que foi o caso da Q1.1 de 10G.

Já na análise do *Apache Hudi*, para dados de 1G executados via *PrestoDB* as consultas Q1.1, Q1.2, Q1.3, Q2.1, Q3.2, Q3.3 e Q4.1 obtiveram uma redução de 1 segundo na leitura após a escrita para a leitura após a atualização. Já os dados de 10G tiveram o mesmo comportamento crescente de tempo em relação a complexidade das consultas, isso pode acontecer devido aos copiar-ao-escrever que aumenta o tempo de escrita e leitura para garantir a consistência dos arquivos e não lida muito bem com arquivos pequenos. Via Athena, os dados de ambas as volumetrias foram bem parecidos, com uma variação de 1 segundo de diferença entre eles.

O *Apache Iceberg* após a atualização demonstrou a constância de 4 segundos nas treze consultas, o que significa que a complexidade das consultas não são um problema na hora de ler os dados. E outro ponto é diminuição de tempo pela metade da Q1.1, mostrando como o *Apache Iceberg* lida com alta volumetria.

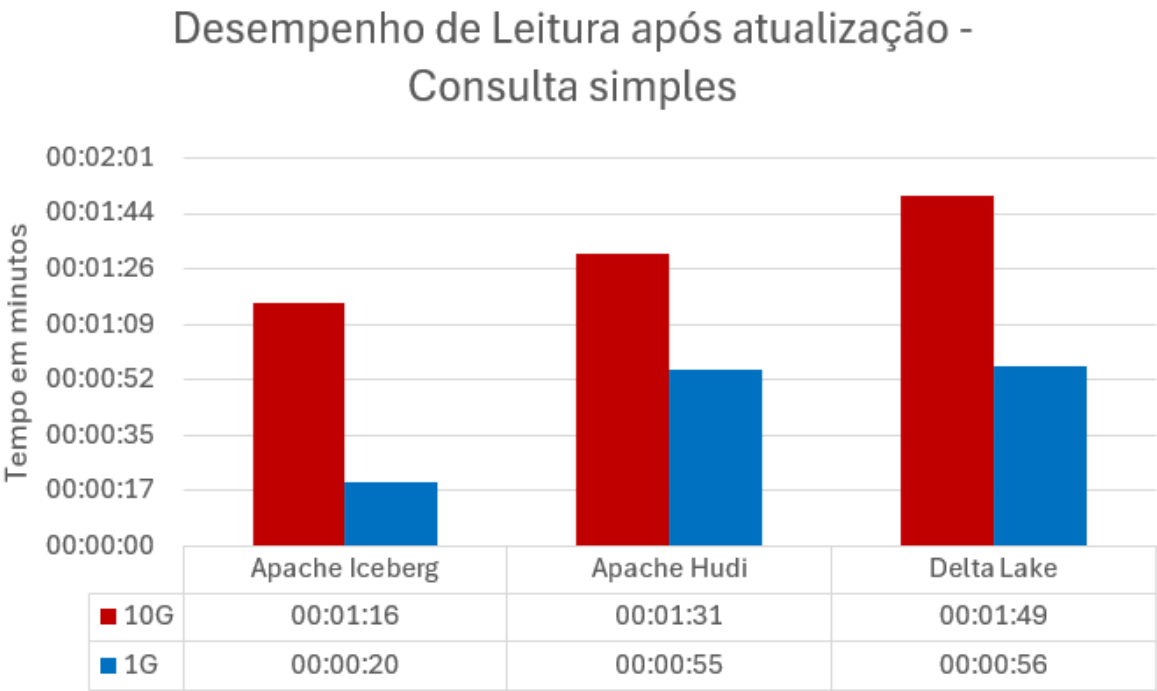
Foi analisado também uma consulta com todos os campos da tabela, sem nenhuma junção, filtros e cálculos no Athena para capturar o tempo de leitura das três tecnologias. Os resultados da consulta simples após a escrita dos dados são apresentados na Figura 19 enquanto os resultados da leitura após consulta são apresentados na Figura 20.

Figura 19 – Resultado de desempenho de Leitura após escrita com consulta simples



Fonte: Elaborado pela Autora (2024)

Figura 20 – Resultado de desempenho de Leitura após atualização com consulta simples



Fonte: Elaborado pela Autora (2024)

Em uma consulta simples, observa-se que, na leitura após a escrita, o *Apache Iceberg* apresentou melhor desempenho com 21 segundos para os dados de 1GB. No entanto, ao aumentar o volume de dados, o *Delta Lake* superou o *Apache Iceberg*, sendo 15 segundos (20,2%) mais rápido. Já na leitura após a atualização, tanto para os dados de 1GB quanto de 10GB, o *Delta Lake* se destacou, apresentando tempos de leitura mais rápidos do que na leitura após a escrita, com uma diferença de 17% para 1GB e 1,25% para 10GB. Os dados granulares de todas as consultas são apresentados na seção APÊNDICE A.2. e A.3.

5 CONCLUSÃO

O experimento realizado evidenciou as vantagens e limitações de cada uma das três tecnologias de armazenamento de dados no contexto de operações de escrita, leitura e atualização em ambientes de dados em grande escala e *Data Lakehouse*. As tecnologias *Apache Hudi*, *Delta Lake* e *Apache Iceberg* apresentaram desempenhos distintos, refletindo as diferenças em suas arquiteturas e estratégias de gerenciamento de dados.

O *Apache Iceberg* se destacou como a solução mais eficiente em termos de escrita e atualização, especialmente quando se trabalhou com volumes maiores de dados. A arquitetura de metadados altamente otimizada e a compactação automática de arquivos pequenos permitiram ao *Apache Iceberg* manter um bom desempenho, mesmo com grandes volumes de dados, o que o torna uma excelente escolha para cenários de *Data Lakehouse* que exigem alta eficiência em operações de leitura e escrita. Seu desempenho consistentemente superior, mesmo com o aumento do volume de dados, reflete sua capacidade de gerenciar eficientemente arquivos pequenos e otimizar a utilização de recursos.

Por outro lado, o *Delta Lake* também demonstrou um bom desempenho geral, mas apresentou aumentos mais acentuados nos tempos de escrita e atualização conforme o volume de dados cresceu. Isso pode ser atribuído à sobrecarga associada à reescrita de arquivos durante transações ACID, especialmente quando se trabalha com arquivos pequenos. No entanto, o *Delta Lake* continuou a se destacar em operações de leitura via Athena.

O *Apache Hudi*, utilizando o modelo *Copy-on-Write* (COW), obteve um desempenho inferior nas operações de escrita e atualização, especialmente em volumes maiores de dados. A reescrita de arquivos completos e a sobrecarga de compactação de arquivos pequenos impactaram significativamente seu desempenho. No entanto, o *Apache Hudi* apresentou resultados superiores na leitura após atualização para volumes menores de dados (1GB), devido aos seus mecanismos de indexação que

otimizam a busca por registros atualizados, o que permite localizar rapidamente as áreas alteradas sem percorrer toda a tabela.

Esses resultados demonstram que a escolha da tecnologia de armazenamento mais adequada depende do caso de uso específico. Para cenários que envolvem grandes volumes de dados e a necessidade de operações rápidas de leitura e escrita, o *Apache Iceberg* se mostrou a melhor opção. O *Delta Lake*, por sua vez, é uma escolha sólida quando se busca transações ACID e consistência de dados, especialmente para dados menores e bem particionados. Já o *Apache Hudi*, com sua estratégia de indexação, se destaca em cenários onde a leitura após atualização de dados é uma prioridade, mas pode não ser a melhor opção quando se trata de grandes volumes e operações intensivas de escrita e atualização.

5.1 Contribuições do trabalho

A contribuição deste trabalho está na análise do impacto das atualizações de dados com diferentes volumes, comparando o desempenho das tecnologias *Apache Hudi*, *Delta Lake* e *Apache Iceberg*. Nesse trabalho foi explorado como o aumento da volumetria, de 1GB para 10GB, afeta o tempo de atualização e leitura dos dados, gerando dados sobre o desempenho de cada tecnologia em cenários de grande escala. Esses resultados são importantes para a escolha de ferramentas adequadas em projetos que exigem alto desempenho para operações de escrita e atualização de dados.

Outra contribuição deste trabalho é a avaliação comparativa das ferramentas *PrestoDB* e *Athena* em diferentes cenários de leitura após a escrita e após a atualização dos dados. A análise desses motores de execução permitiu identificar as vantagens e limitações de cada plataforma, oferecendo mais informações para a tomada de decisão na escolha da melhor ferramenta para consultas em ambientes *Data Lakehouses*. Esse teste pode ser especialmente útil para organizações que buscam otimizar o desempenho de suas consultas em grandes volumes de dados.

5.2 Trabalhos futuros

Para trabalhos futuros, seria interessante expandir os testes realizando comparações entre os motores de execução *Spark* e *PrestoDB*, explorando o desempenho em diferentes cenários de volumetria e complexidade de consulta (junções, filtros, união de dados, ordenação e etc). Além disso, uma análise em volumes de dados maiores, como 1000G, poderia fornecer mais percepções detalhadas sobre a escalabilidade das tecnologias estudadas. Também seria relevante investigar o impacto de operações de manutenção, como *vacuum*, *clean* e *optimize*, a fim de avaliar como essas operações afetam o desempenho e a eficiência do armazenamento ao longo do tempo. Outra área promissora para investigação seria a aplicação de particionamento e indexação, visando otimizar as consultas e melhorar o tempo de resposta em grandes volumes de dados e também evitar arquivos pequenos e analisar o comportamento do *Apache Hudi*.

Um outro ponto importante é resolver o erro ao tentar ler os dados da tabela *Apache Iceberg* via *PrestoDB*, assim resultaria em uma comparação justa entre as três tecnologias com esse motor de execução.

Além disso, uma análise do *Delta Universal Format* (UniForm) seria uma contribuição importante para a evolução dos testes, explorando como essa opção pode impactar a compatibilidade e o desempenho das operações de escrita e leitura. Por fim, uma extensão dos testes para diferentes provedores de nuvem e outros mecanismos de SQL, como o *Google Cloud Platform* (GCP) e o *BigQuery*, permitiria avaliar as ferramentas sob diferentes arquiteturas e ambientes.

REFERÊNCIAS BIBLIOGRÁFICA

- Ait Errami et al. (2023) Soukaina Ait Errami et al. **Spatial big data architecture: From Data Warehouses and Data Lakes to the LakeHouse**. *Journal of Parallel and Distributed Computing*. 176, (Jun.-2023), 70–79. doi: 10.1016/j.jpdc.2023.02.007.
- AYDOĞAN e KARCI (2018) Murat AYDOĞAN and Ali KARCI. **Spam Mail Detection using Naive Bayes method with Apache Spark**. *2018 International Conference on Artificial Intelligence and Data Processing (IDAP)* (Sep.-2018), 1–6. doi: 10.1109/IDAP.2018.8620737.
- Belov e Nikulchev (2021) Vladimir Belov and Evgeny Nikulchev. **Analysis of Big Data Storage Tools for Data Lakes based on Apache Hadoop Platform**. *International Journal of Advanced Computer Science and Applications*. 12, (Aug.-2021), 551–557. doi: 10.14569/IJACSA.2021.0120864.
- Chae e Chung (2019) Suk-Joo Chae and Tae-Sun Chung. **DSMM: A Dynamic Setting for Memory Management in Apache Spark**. *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)* (Mar.-2019), 143–144. doi: 10.1109/ISPASS.2019.00024.
- Chaudhuri e Dayal (1997) Surajit Chaudhuri and Umeshwar Dayal. **An overview of data warehousing and OLAP technology**. *SIGMOD Rec.* 26, 1 (Mar.-1997), 65–74. doi: 10.1145/248603.248616.
- Čuš e Golec (2023) Blaž Čuš and Darko Golec. **Data Lakehouse: Benefits in small and medium enterprises**. *Mednarodno inovativno poslovanje = Journal of Innovative Business and Management*. 14, 2 (Apr.-2023), 1–10. doi: 10.32015/JIBM.2022.14.2.5.
- Cuzzocrea (2021) Alfredo Cuzzocrea. **Big Data Lakes: Models, Frameworks, and Techniques**. *2021 IEEE International Conference on Big Data and Smart Computing (BigComp)* (Jan.-2021), 1–4. doi: 10.1109/BigComp51126.2021.00010.
- “Delta Lake for ETL” (2024) <https://delta.io/blog/delta-lake-etl/>. Accessed: 03-Nov.-2024. Retrieved 3-Nov.-2024, from <https://delta.io/blog/delta-lake-etl/>.
- Fang (2015) Huang Fang. **Managing data lakes in big data era: What’s a data lake and why has it become popular in data management ecosystem**. *2015 IEEE International Conference on Cyber Technology in Automation, Control, and Intelligent Systems (CYBER)* (Jun.-2015), 820–824. doi: 10.1109/CYBER.2015.7288049.

- Harby e Zulkernine (2022) Ahmed A. Harby and Farhana Zulkernine. **From Data Warehouse to Lakehouse: A Comparative Review**. *2022 IEEE International Conference on Big Data (Big Data)* (Dec.-2022), 389–395. doi: 10.1109/BigData55660.2022.10020719.
- Inmon (2005) W. H. Inmon. **Building the Data Warehouse**. John Wiley & Sons.
- Jain et al. (2023) Paras Jain et al. **Analyzing and Comparing Lakehouse Storage Systems**. (2023).
- Karthikeya Sahith et al. (2023) Chaganti Sri Karthikeya Sahith et al. **Apache Spark Big data Analysis, Performance Tuning, and Spark Application Optimization**. *2023 International Conference on Evolutionary Algorithms and Soft Computing Techniques (EASCT)* (Oct.-2023), 1–8. doi: 10.1109/EASCT59475.2023.10393086.
- Kumar e Kavita (2019) Kumar e Kavita. **<https://www.scribd.com/document/566881039/jvp42019>**. Accessed: 23-Nov.-2024. Retrieved 23-Nov.-2024, from <https://www.scribd.com/document/566881039/jvp42019>.
- Moody e Kortink (2000) D. Moody and M. Kortink. **From enterprise models to dimensional models: a methodology for data warehouse and data mart design**. (2000). Retrieved 23-Nov.-2024, from <https://www.semanticscholar.org/paper/From-enterprise-models-to-dimensional-models%3A-a-for-Moody-Kortink/a2a2d7bb089fcfaa95e9703bb6d38b286f071415>.
- Orescanin e Hlupic (2021) Drazen Orescanin and Tomislav Hlupic. **Data Lakehouse - A Novel Step in Analytics Architecture**. *2021 44th International Convention on Information, Communication and Electronic Technology, MIPRO 2021 - Proceedings*. (2021), 1242–1246. doi: 10.23919/MIPRO52101.2021.9597091.
- Oussous et al. (2018) Ahmed Oussous et al. **Big Data technologies: A survey**. *Journal of King Saud University - Computer and Information Sciences*. 30, 4 (Oct.-2018), 431–448. doi: 10.1016/j.jksuci.2017.06.001.
- Raslan e Calazans (2014) Daniela Andrade Raslan and Angélica Toffano Seidel Calazans. **Data Warehouse: conceitos e aplicações** - DOI: 10.5102/un.gti.v4i1.2612. *Universitas: Gestão e TI*. 4, 1 (Aug.-2014). doi: 10.5102/un.gti.v4i1.2612.
- Sethi et al. (2019) Raghav Sethi et al. **Presto: SQL on Everything**. *2019 IEEE 35th International Conference on Data Engineering (ICDE)* (Apr.-2019), 1802–1813. doi: 10.1109/ICDE.2019.00196.

- Singh et al. (2022) Jaspreet Singh et al. **The Implication of Data Lake in Enterprises: A Deeper Analytics**. *2022 8th International Conference on Advanced Computing and Communication Systems (ICACCS)* (Mar.-2022), 530–534. doi: 10.1109/ICACCS54159.2022.9784986.
- Tang et al. (2024) Xin Tang et al. **Separation Is for Better Reunion: Data Lake Storage at Huawei**. *2024 IEEE 40th International Conference on Data Engineering (ICDE)* (May.-2024), 5142–5155. doi: 10.1109/ICDE60146.2024.00386.
- Tardío et al. (2022) Roberto Tardío et al. **Beyond TPC-DS, a benchmark for Big Data OLAP systems (BDOLAP-Bench)**. *Future Generation Computer Systems*. 132, (Jul.-2022), 136–151. doi: 10.1016/j.future.2022.02.015.
- “What is a Data Lakehouse & How does it Work?” (2024) <https://hudi.apache.org/blog/2024/07/11/what-is-a-data-lakehouse>. Accessed: 02-Nov.-2024. Retrieved 2-Nov.-2024, from <https://hudi.apache.org/blog/2024/07/11/what-is-a-data-lakehouse>.
- Yessad e Labiod (2016) Lamia Yessad and Aissa Labiod. **Comparative study of data warehouses modeling approaches: Inmon, Kimball and Data Vault**. *2016 International Conference on System Reliability and Science (ICSRS)* (Paris, France, Nov.-2016), 95–99. doi: 10.1109/ICSRS.2016.7815845.
- Zhang et al. (2023) Tao Zhang et al. **Corddl: An Efficient and Extensible Connector between Relational Databases and Data Lakes**. *2023 5th International Conference on Machine Learning, Big Data and Business Intelligence (MLBDBI)* (Dec.-2023), 173–177. doi: 10.1109/MLBDBI60823.2023.10481929.

APÊNDICE A

A.1. Consultas disponibilizadas pelo SSB

Q1.1

```
select sum(v_revenue) as revenue
from p_lineorder
left join dates on lo_orderdate = d_datekey
where d_year = 1993
and lo_discount between 1 and 3
and lo_quantity < 25;
```

Q1.2

```
select sum(v_revenue) as revenue
from p_lineorder
left join dates on lo_orderdate = d_datekey
where d_yearmonthnum = 199401
and lo_discount between 4 and 6
and lo_quantity between 26 and 35;
```

Q1.3

```
select sum(v_revenue) as revenue
from p_lineorder
left join dates on lo_orderdate = d_datekey
where d_weeknuminyear = 6 and d_year = 1994
and lo_discount between 5 and 7
and lo_quantity between 26 and 35;
```

Q2.1

```
select sum(lo_revenue) as lo_revenue, d_year, p_brand
from p_lineorder
left join dates on lo_orderdate = d_datekey
left join part on lo_partkey = p_partkey
left join supplier on lo_suppkey = s_suppkey
where p_category = 'MFGR#12' and s_region = 'AMERICA'
group by d_year, p_brand
order by d_year, p_brand;
```

Q2.2

```
select sum(lo_revenue) as lo_revenue, d_year, p_brand
from p_lineorder
left join dates on lo_orderdate = d_datekey
left join part on lo_partkey = p_partkey
left join supplier on lo_suppkey = s_suppkey
where p_brand between 'MFGR#2221' and 'MFGR#2228' and s_region = 'ASIA'
```

```
group by d_year, p_brand
order by d_year, p_brand;
```

Q2.3

```
select sum(lo_revenue) as lo_revenue, d_year, p_brand
from p_lineorder
left join dates on lo_orderdate = d_datekey
left join part on lo_partkey = p_partkey
left join supplier on lo_suppkey = s_suppkey
where p_brand = 'MFGR#2239' and s_region = 'EUROPE'
group by d_year, p_brand
order by d_year, p_brand;
```

Q3.1

```
select c_nation, s_nation, d_year, sum(lo_revenue) as lo_revenue
from p_lineorder
left join dates on lo_orderdate = d_datekey
left join customer on lo_custkey = c_custkey
left join supplier on lo_suppkey = s_suppkey
where c_region = 'ASIA' and s_region = 'ASIA' and d_year >= 1992 and d_year
<= 1997
group by c_nation, s_nation, d_year
order by d_year asc, lo_revenue desc;
```

Q3.2

```
select c_city, s_city, d_year, sum(lo_revenue) as lo_revenue
from p_lineorder
left join dates on lo_orderdate = d_datekey
left join customer on lo_custkey = c_custkey
left join supplier on lo_suppkey = s_suppkey
where c_nation = 'UNITED STATES' and s_nation = 'UNITED STATES'
and d_year >= 1992 and d_year <= 1997
group by c_city, s_city, d_year
order by d_year asc, lo_revenue desc;
```

Q3.3

```
select c_city, s_city, d_year, sum(lo_revenue) as lo_revenue
from p_lineorder
left join dates on lo_orderdate = d_datekey
left join customer on lo_custkey = c_custkey
left join supplier on lo_suppkey = s_suppkey
where (c_city='UNITED KI1' or c_city='UNITED KI5')
and (s_city='UNITED KI1' or s_city='UNITED KI5')
and d_year >= 1992 and d_year <= 1997
group by c_city, s_city, d_year
order by d_year asc, lo_revenue desc;
```

Q3.4

```
select c_city, s_city, d_year, sum(lo_revenue) as lo_revenue
```

```

from p_lineorder
left join dates on lo_orderdate = d_datekey
left join customer on lo_custkey = c_custkey
left join supplier on lo_suppkey = s_suppkey
where (c_city='UNITED KI1' or c_city='UNITED KI5') and (s_city='UNITED KI1'
or s_city='UNITED KI5') and d_yearmonth = 'Dec1997'
group by c_city, s_city, d_year
order by d_year asc, lo_revenue desc;

```

Q4.1

```

select d_year, c_nation, sum(lo_revenue) - sum(lo_supplycost) as profit
from p_lineorder
left join dates on lo_orderdate = d_datekey
left join customer on lo_custkey = c_custkey
left join supplier on lo_suppkey = s_suppkey
left join part on lo_partkey = p_partkey
where c_region = 'AMERICA' and s_region = 'AMERICA' and (p_mfgr = 'MFGR#1'
or p_mfgr = 'MFGR#2')
group by d_year, c_nation
order by d_year, c_nation;

```

Q4.2

```

select d_year, s_nation, p_category, sum(lo_revenue) - sum(lo_supplycost)
as profit
from p_lineorder
left join dates on lo_orderdate = d_datekey
left join customer on lo_custkey = c_custkey
left join supplier on lo_suppkey = s_suppkey
left join part on lo_partkey = p_partkey
where c_region = 'AMERICA' and s_region = 'AMERICA'
and (d_year = 1997 or d_year = 1998)
and (p_mfgr = 'MFGR#1' or p_mfgr = 'MFGR#2')
group by d_year, s_nation, p_category
order by d_year, s_nation, p_category;

```

Q4.3

```

select d_year, s_city, p_brand, sum(lo_revenue) - sum(lo_supplycost) as
profit
from p_lineorder
left join dates on lo_orderdate = d_datekey
left join customer on lo_custkey = c_custkey
left join supplier on lo_suppkey = s_suppkey
left join part on lo_partkey = p_partkey
where c_region = 'AMERICA' and s_nation = 'UNITED STATES'
and (d_year = 1997 or d_year = 1998)
and p_category = 'MFGR#14'
group by d_year, s_city, p_brand
order by d_year, s_city, p_brand;

```

A.2. Dados granulares dos resultados

A.2.1. Escrita

Tecnologia	Operação	Tamanho	Tempo	Mecanismo
Delta Lake	Escrita	1G	00:01:16	Spark
Apache Hudi	Escrita	1G	00:06:27	Spark
Apache Iceberg	Escrita	1G	00:01:05	Spark
Delta Lake	Escrita	10G	00:02:18	Spark
Apache Hudi	Escrita	10G	00:12:09	Spark
Apache Iceberg	Escrita	10G	00:01:28	Spark

A.2.2. Atualização

Tecnologia	Operação	Tamanho	Tempo	Engine
Delta Lake	Atualização	1G	00:03:19	Spark
Apache Hudi	Atualização	1G	00:04:42	Spark
Apache Iceberg	Atualização	1G	00:01:19	Spark
Delta Lake	Atualização	10G	00:04:17	Spark
Apache Hudi	Atualização	10G	00:09:10	Spark
Apache Iceberg	Atualização	10G	00:02:34	Spark

A.2.3. Leitura após a escrita

Tecnologia	Operação	Tamanho	Tempo	Query	Engine
Apache Hudi	Leitura_escrita	1G	00:00:04	Q1.1	Presto
Apache Hudi	Leitura_escrita	1G	00:00:04	Q1.2	Presto
Apache Hudi	Leitura_escrita	1G	00:00:05	Q1.3	Presto
Apache Hudi	Leitura_escrita	1G	00:00:06	Q2.1	Presto
Apache Hudi	Leitura_escrita	1G	00:00:06	Q2.2	Presto
Apache Hudi	Leitura_escrita	1G	00:00:06	Q2.3	Presto
Apache Hudi	Leitura_escrita	1G	00:00:06	Q3.1	Presto
Apache Hudi	Leitura_escrita	1G	00:00:06	Q3.2	Presto
Apache Hudi	Leitura_escrita	1G	00:00:06	Q3.3	Presto
Apache Hudi	Leitura_escrita	1G	00:00:06	Q3.4	Presto
Apache Hudi	Leitura_escrita	1G	00:00:07	Q4.1	Presto

Apache Hudi	Leitura_escrita	1G	00:00:07	Q4.2	Presto
Apache Hudi	Leitura_escrita	1G	00:00:07	Q4.3	Presto
Delta Lake	Leitura_escrita	1G	00:00:05	Q1.1	Presto
Delta Lake	Leitura_escrita	1G	00:00:05	Q1.2	Presto
Delta Lake	Leitura_escrita	1G	00:00:04	Q1.3	Presto
Delta Lake	Leitura_escrita	1G	00:00:04	Q2.1	Presto
Delta Lake	Leitura_escrita	1G	00:00:05	Q2.2	Presto
Delta Lake	Leitura_escrita	1G	00:00:05	Q2.3	Presto
Delta Lake	Leitura_escrita	1G	00:00:05	Q3.1	Presto
Delta Lake	Leitura_escrita	1G	00:00:08	Q3.2	Presto
Delta Lake	Leitura_escrita	1G	00:00:04	Q3.3	Presto
Delta Lake	Leitura_escrita	1G	00:00:04	Q3.4	Presto
Delta Lake	Leitura_escrita	1G	00:00:10	Q4.1	Presto
Delta Lake	Leitura_escrita	1G	00:00:05	Q4.2	Presto
Delta Lake	Leitura_escrita	1G	00:00:08	Q4.3	Presto
Delta Lake	Leitura_escrita	1G	00:00:02	Q1.1	Athena
Delta Lake	Leitura_escrita	1G	00:00:01	Q1.2	Athena
Delta Lake	Leitura_escrita	1G	00:00:02	Q1.3	Athena
Delta Lake	Leitura_escrita	1G	00:00:02	Q2.1	Athena
Delta Lake	Leitura_escrita	1G	00:00:02	Q2.2	Athena
Delta Lake	Leitura_escrita	1G	00:00:02	Q2.3	Athena
Delta Lake	Leitura_escrita	1G	00:00:02	Q3.1	Athena
Delta Lake	Leitura_escrita	1G	00:00:02	Q3.2	Athena
Delta Lake	Leitura_escrita	1G	00:00:02	Q3.3	Athena
Delta Lake	Leitura_escrita	1G	00:00:02	Q3.4	Athena
Delta Lake	Leitura_escrita	1G	00:00:03	Q4.1	Athena
Delta Lake	Leitura_escrita	1G	00:00:03	Q4.2	Athena
Delta Lake	Leitura_escrita	1G	00:00:02	Q4.3	Athena
Apache Hudi	Leitura_escrita	1G	00:00:02	Q1.1	Athena
Apache Hudi	Leitura_escrita	1G	00:00:02	Q1.2	Athena
Apache Hudi	Leitura_escrita	1G	00:00:02	Q1.3	Athena
Apache Hudi	Leitura_escrita	1G	00:00:03	Q2.1	Athena
Apache Hudi	Leitura_escrita	1G	00:00:03	Q2.2	Athena
Apache Hudi	Leitura_escrita	1G	00:00:02	Q2.3	Athena
Apache Hudi	Leitura_escrita	1G	00:00:02	Q3.1	Athena
Apache Hudi	Leitura_escrita	1G	00:00:03	Q3.2	Athena
Apache Hudi	Leitura_escrita	1G	00:00:03	Q3.3	Athena
Apache Hudi	Leitura_escrita	1G	00:00:02	Q3.4	Athena
Apache Hudi	Leitura_escrita	1G	00:00:02	Q4.1	Athena
Apache Hudi	Leitura_escrita	1G	00:00:02	Q4.2	Athena
Apache Hudi	Leitura_escrita	1G	00:00:03	Q4.3	Athena

Apache Iceberg	Leitura_escrita	1G	00:00:04	Q1.1	Athena
Apache Iceberg	Leitura_escrita	1G	00:00:03	Q1.2	Athena
Apache Iceberg	Leitura_escrita	1G	00:00:04	Q1.3	Athena
Apache Iceberg	Leitura_escrita	1G	00:00:04	Q2.1	Athena
Apache Iceberg	Leitura_escrita	1G	00:00:04	Q2.2	Athena
Apache Iceberg	Leitura_escrita	1G	00:00:04	Q2.3	Athena
Apache Iceberg	Leitura_escrita	1G	00:00:05	Q3.1	Athena
Apache Iceberg	Leitura_escrita	1G	00:00:04	Q3.2	Athena
Apache Iceberg	Leitura_escrita	1G	00:00:04	Q3.3	Athena
Apache Iceberg	Leitura_escrita	1G	00:00:04	Q3.4	Athena
Apache Iceberg	Leitura_escrita	1G	00:00:04	Q4.1	Athena
Apache Iceberg	Leitura_escrita	1G	00:00:04	Q4.2	Athena
Apache Iceberg	Leitura_escrita	1G	00:00:03	Q4.3	Athena
Delta Lake	Leitura_escrita	10G	00:00:05	Q1.1	Presto
Delta Lake	Leitura_escrita	10G	00:00:03	Q1.2	Presto
Delta Lake	Leitura_escrita	10G	00:00:04	Q1.3	Presto
Delta Lake	Leitura_escrita	10G	00:00:06	Q2.1	Presto
Delta Lake	Leitura_escrita	10G	00:00:06	Q2.2	Presto
Delta Lake	Leitura_escrita	10G	00:00:05	Q2.3	Presto
Delta Lake	Leitura_escrita	10G	00:00:05	Q3.1	Presto
Delta Lake	Leitura_escrita	10G	00:00:05	Q3.2	Presto
Delta Lake	Leitura_escrita	10G	00:00:04	Q3.3	Presto
Delta Lake	Leitura_escrita	10G	00:00:05	Q3.4	Presto
Delta Lake	Leitura_escrita	10G	00:00:07	Q4.1	Presto
Delta Lake	Leitura_escrita	10G	00:00:07	Q4.2	Presto
Delta Lake	Leitura_escrita	10G	00:00:07	Q4.3	Presto
Apache Hudi	Leitura_escrita	10G	00:00:06	Q1.1	Presto
Apache Hudi	Leitura_escrita	10G	00:00:06	Q1.2	Presto
Apache Hudi	Leitura_escrita	10G	00:00:06	Q1.3	Presto
Apache Hudi	Leitura_escrita	10G	00:00:08	Q2.1	Presto
Apache Hudi	Leitura_escrita	10G	00:00:08	Q2.2	Presto
Apache Hudi	Leitura_escrita	10G	00:00:08	Q2.3	Presto
Apache Hudi	Leitura_escrita	10G	00:00:08	Q3.1	Presto
Apache Hudi	Leitura_escrita	10G	00:00:07	Q3.2	Presto
Apache Hudi	Leitura_escrita	10G	00:00:08	Q3.3	Presto
Apache Hudi	Leitura_escrita	10G	00:00:07	Q3.4	Presto
Apache Hudi	Leitura_escrita	10G	00:00:10	Q4.1	Presto
Apache Hudi	Leitura_escrita	10G	00:00:10	Q4.2	Presto
Apache Hudi	Leitura_escrita	10G	00:00:10	Q4.3	Presto
Delta Lake	Leitura_escrita	10G	00:00:02	Q1.1	Athena
Delta Lake	Leitura_escrita	10G	00:00:02	Q1.2	Athena

Delta Lake	Leitura_escrita	10G	00:00:02	Q1.3	Athena
Delta Lake	Leitura_escrita	10G	00:00:02	Q2.1	Athena
Delta Lake	Leitura_escrita	10G	00:00:02	Q2.2	Athena
Delta Lake	Leitura_escrita	10G	00:00:02	Q2.3	Athena
Delta Lake	Leitura_escrita	10G	00:00:03	Q3.1	Athena
Delta Lake	Leitura_escrita	10G	00:00:03	Q3.2	Athena
Delta Lake	Leitura_escrita	10G	00:00:02	Q3.3	Athena
Delta Lake	Leitura_escrita	10G	00:00:02	Q3.4	Athena
Delta Lake	Leitura_escrita	10G	00:00:03	Q4.1	Athena
Delta Lake	Leitura_escrita	10G	00:00:03	Q4.2	Athena
Delta Lake	Leitura_escrita	10G	00:00:03	Q4.3	Athena
Apache Hudi	Leitura_escrita	10G	00:00:04	Q1.1	Athena
Apache Hudi	Leitura_escrita	10G	00:00:04	Q1.2	Athena
Apache Hudi	Leitura_escrita	10G	00:00:04	Q1.3	Athena
Apache Hudi	Leitura_escrita	10G	00:00:04	Q2.1	Athena
Apache Hudi	Leitura_escrita	10G	00:00:05	Q2.2	Athena
Apache Hudi	Leitura_escrita	10G	00:00:05	Q2.3	Athena
Apache Hudi	Leitura_escrita	10G	00:00:05	Q3.1	Athena
Apache Hudi	Leitura_escrita	10G	00:00:06	Q3.2	Athena
Apache Hudi	Leitura_escrita	10G	00:00:06	Q3.3	Athena
Apache Hudi	Leitura_escrita	10G	00:00:06	Q3.4	Athena
Apache Hudi	Leitura_escrita	10G	00:00:06	Q4.1	Athena
Apache Hudi	Leitura_escrita	10G	00:00:06	Q4.2	Athena
Apache Hudi	Leitura_escrita	10G	00:00:06	Q4.3	Athena
Apache Iceberg	Leitura_escrita	10G	00:00:04	Q1.1	Athena
Apache Iceberg	Leitura_escrita	10G	00:00:03	Q1.2	Athena
Apache Iceberg	Leitura_escrita	10G	00:00:04	Q1.3	Athena
Apache Iceberg	Leitura_escrita	10G	00:00:04	Q2.1	Athena
Apache Iceberg	Leitura_escrita	10G	00:00:04	Q2.2	Athena
Apache Iceberg	Leitura_escrita	10G	00:00:04	Q2.3	Athena
Apache Iceberg	Leitura_escrita	10G	00:00:05	Q3.1	Athena
Apache Iceberg	Leitura_escrita	10G	00:00:04	Q3.2	Athena
Apache Iceberg	Leitura_escrita	10G	00:00:04	Q3.3	Athena
Apache Iceberg	Leitura_escrita	10G	00:00:04	Q3.4	Athena
Apache Iceberg	Leitura_escrita	10G	00:00:04	Q4.1	Athena
Apache Iceberg	Leitura_escrita	10G	00:00:04	Q4.2	Athena
Apache Iceberg	Leitura_escrita	10G	00:00:03	Q4.3	Athena

A.2.4. Leitura após a atualização

Tecnologia	Operação	Tamanho	Tempo	Query	Engine
Apache Hudi	Leitura_atualização	1G	00:00:03	Q1.1	Presto
Apache Hudi	Leitura_atualização	1G	00:00:03	Q1.2	Presto
Apache Hudi	Leitura_atualização	1G	00:00:03	Q1.3	Presto
Apache Hudi	Leitura_atualização	1G	00:00:05	Q2.1	Presto
Apache Hudi	Leitura_atualização	1G	00:00:06	Q2.2	Presto
Apache Hudi	Leitura_atualização	1G	00:00:05	Q2.3	Presto
Apache Hudi	Leitura_atualização	1G	00:00:06	Q3.1	Presto
Apache Hudi	Leitura_atualização	1G	00:00:05	Q3.2	Presto
Apache Hudi	Leitura_atualização	1G	00:00:05	Q3.3	Presto
Apache Hudi	Leitura_atualização	1G	00:00:06	Q3.4	Presto
Apache Hudi	Leitura_atualização	1G	00:00:06	Q4.1	Presto
Apache Hudi	Leitura_atualização	1G	00:00:07	Q4.2	Presto
Apache Hudi	Leitura_atualização	1G	00:00:07	Q4.3	Presto
Delta Lake	Leitura_atualização	1G	00:00:08	Q1.1	Presto
Delta Lake	Leitura_atualização	1G	00:00:07	Q1.2	Presto
Delta Lake	Leitura_atualização	1G	00:00:07	Q1.3	Presto
Delta Lake	Leitura_atualização	1G	00:00:11	Q2.1	Presto
Delta Lake	Leitura_atualização	1G	00:00:08	Q2.2	Presto
Delta Lake	Leitura_atualização	1G	00:00:07	Q2.3	Presto
Delta Lake	Leitura_atualização	1G	00:00:10	Q3.1	Presto
Delta Lake	Leitura_atualização	1G	00:00:07	Q3.2	Presto
Delta Lake	Leitura_atualização	1G	00:00:08	Q3.3	Presto
Delta Lake	Leitura_atualização	1G	00:00:08	Q3.4	Presto
Delta Lake	Leitura_atualização	1G	00:00:11	Q4.1	Presto
Delta Lake	Leitura_atualização	1G	00:00:12	Q4.2	Presto
Delta Lake	Leitura_atualização	1G	00:00:10	Q4.3	Presto
Delta Lake	Leitura_atualização	1G	00:00:02	Q1.1	Athena
Delta Lake	Leitura_atualização	1G	00:00:02	Q1.2	Athena
Delta Lake	Leitura_atualização	1G	00:00:03	Q1.3	Athena
Delta Lake	Leitura_atualização	1G	00:00:02	Q2.1	Athena
Delta Lake	Leitura_atualização	1G	00:00:02	Q2.2	Athena
Delta Lake	Leitura_atualização	1G	00:00:03	Q2.3	Athena
Delta Lake	Leitura_atualização	1G	00:00:03	Q3.1	Athena
Delta Lake	Leitura_atualização	1G	00:00:03	Q3.2	Athena
Delta Lake	Leitura_atualização	1G	00:00:03	Q3.3	Athena
Delta Lake	Leitura_atualização	1G	00:00:03	Q3.4	Athena
Delta Lake	Leitura_atualização	1G	00:00:03	Q4.1	Athena
Delta Lake	Leitura_atualização	1G	00:00:03	Q4.2	Athena

Delta Lake	Leitura_atualização	1G	00:00:03	Q4.3	Athena
Apache Hudi	Leitura_atualização	1G	00:00:03	Q1.1	Athena
Apache Hudi	Leitura_atualização	1G	00:00:03	Q1.2	Athena
Apache Hudi	Leitura_atualização	1G	00:00:03	Q1.3	Athena
Apache Hudi	Leitura_atualização	1G	00:00:04	Q2.1	Athena
Apache Hudi	Leitura_atualização	1G	00:00:05	Q2.2	Athena
Apache Hudi	Leitura_atualização	1G	00:00:04	Q2.3	Athena
Apache Hudi	Leitura_atualização	1G	00:00:05	Q3.1	Athena
Apache Hudi	Leitura_atualização	1G	00:00:04	Q3.2	Athena
Apache Hudi	Leitura_atualização	1G	00:00:05	Q3.3	Athena
Apache Hudi	Leitura_atualização	1G	00:00:05	Q3.4	Athena
Apache Hudi	Leitura_atualização	1G	00:00:05	Q4.1	Athena
Apache Hudi	Leitura_atualização	1G	00:00:05	Q4.2	Athena
Apache Hudi	Leitura_atualização	1G	00:00:05	Q4.3	Athena
Apache Iceberg	Leitura_atualização	1G	00:00:04	Q1.1	Athena
Apache Iceberg	Leitura_atualização	1G	00:00:04	Q1.2	Athena
Apache Iceberg	Leitura_atualização	1G	00:00:04	Q1.3	Athena
Apache Iceberg	Leitura_atualização	1G	00:00:04	Q2.1	Athena
Apache Iceberg	Leitura_atualização	1G	00:00:04	Q2.2	Athena
Apache Iceberg	Leitura_atualização	1G	00:00:04	Q2.3	Athena
Apache Iceberg	Leitura_atualização	1G	00:00:04	Q3.1	Athena
Apache Iceberg	Leitura_atualização	1G	00:00:04	Q3.2	Athena
Apache Iceberg	Leitura_atualização	1G	00:00:04	Q3.3	Athena
Apache Iceberg	Leitura_atualização	1G	00:00:04	Q3.4	Athena
Apache Iceberg	Leitura_atualização	1G	00:00:04	Q4.1	Athena
Apache Iceberg	Leitura_atualização	1G	00:00:04	Q4.2	Athena
Apache Iceberg	Leitura_atualização	1G	00:00:04	Q4.3	Athena
Apache Hudi	Leitura_atualização	10G	00:00:03	Q1.1	Athena
Apache Hudi	Leitura_atualização	10G	00:00:03	Q1.2	Athena
Apache Hudi	Leitura_atualização	10G	00:00:03	Q1.3	Athena
Apache Hudi	Leitura_atualização	10G	00:00:03	Q2.1	Athena
Apache Hudi	Leitura_atualização	10G	00:00:04	Q2.2	Athena
Apache Hudi	Leitura_atualização	10G	00:00:04	Q2.3	Athena
Apache Hudi	Leitura_atualização	10G	00:00:04	Q3.1	Athena
Apache Hudi	Leitura_atualização	10G	00:00:04	Q3.2	Athena
Apache Hudi	Leitura_atualização	10G	00:00:04	Q3.3	Athena
Apache Hudi	Leitura_atualização	10G	00:00:04	Q3.4	Athena
Apache Hudi	Leitura_atualização	10G	00:00:05	Q4.1	Athena
Apache Hudi	Leitura_atualização	10G	00:00:05	Q4.2	Athena
Apache Hudi	Leitura_atualização	10G	00:00:05	Q4.3	Athena
Delta Lake	Leitura_atualização	10G	00:00:03	Q1.1	Athena

Delta Lake	Leitura_atualização	10G	00:00:02	Q1.2	Athena
Delta Lake	Leitura_atualização	10G	00:00:03	Q1.3	Athena
Delta Lake	Leitura_atualização	10G	00:00:03	Q2.1	Athena
Delta Lake	Leitura_atualização	10G	00:00:02	Q2.2	Athena
Delta Lake	Leitura_atualização	10G	00:00:03	Q2.3	Athena
Delta Lake	Leitura_atualização	10G	00:00:04	Q3.1	Athena
Delta Lake	Leitura_atualização	10G	00:00:04	Q3.2	Athena
Delta Lake	Leitura_atualização	10G	00:00:04	Q3.3	Athena
Delta Lake	Leitura_atualização	10G	00:00:04	Q3.4	Athena
Delta Lake	Leitura_atualização	10G	00:00:03	Q4.1	Athena
Delta Lake	Leitura_atualização	10G	00:00:04	Q4.2	Athena
Delta Lake	Leitura_atualização	10G	00:00:04	Q4.3	Athena
Apache Iceberg	Leitura_atualização	10G	00:00:02	Q1.1	Athena
Apache Iceberg	Leitura_atualização	10G	00:00:02	Q1.2	Athena
Apache Iceberg	Leitura_atualização	10G	00:00:03	Q1.3	Athena
Apache Iceberg	Leitura_atualização	10G	00:00:03	Q2.1	Athena
Apache Iceberg	Leitura_atualização	10G	00:00:03	Q2.2	Athena
Apache Iceberg	Leitura_atualização	10G	00:00:04	Q2.3	Athena
Apache Iceberg	Leitura_atualização	10G	00:00:04	Q3.1	Athena
Apache Iceberg	Leitura_atualização	10G	00:00:03	Q3.2	Athena
Apache Iceberg	Leitura_atualização	10G	00:00:04	Q3.3	Athena
Apache Iceberg	Leitura_atualização	10G	00:00:04	Q3.4	Athena
Apache Iceberg	Leitura_atualização	10G	00:00:04	Q4.1	Athena
Apache Iceberg	Leitura_atualização	10G	00:00:04	Q4.2	Athena
Apache Iceberg	Leitura_atualização	10G	00:00:03	Q4.3	Athena
Apache Hudi	Leitura_atualização	10G	00:00:08	Q1.1	Presto
Apache Hudi	Leitura_atualização	10G	00:00:09	Q1.2	Presto
Apache Hudi	Leitura_atualização	10G	00:00:09	Q1.3	Presto
Apache Hudi	Leitura_atualização	10G	00:00:10	Q2.1	Presto
Apache Hudi	Leitura_atualização	10G	00:00:11	Q2.2	Presto
Apache Hudi	Leitura_atualização	10G	00:00:10	Q2.3	Presto
Apache Hudi	Leitura_atualização	10G	00:00:10	Q3.1	Presto
Apache Hudi	Leitura_atualização	10G	00:00:11	Q3.2	Presto
Apache Hudi	Leitura_atualização	10G	00:00:11	Q3.3	Presto
Apache Hudi	Leitura_atualização	10G	00:00:10	Q3.4	Presto
Apache Hudi	Leitura_atualização	10G	00:00:13	Q4.1	Presto
Apache Hudi	Leitura_atualização	10G	00:00:15	Q4.2	Presto
Apache Hudi	Leitura_atualização	10G	00:00:15	Q4.3	Presto
Delta Lake	Leitura_atualização	10G	00:00:06	Q1.1	Presto
Delta Lake	Leitura_atualização	10G	00:00:06	Q1.2	Presto
Delta Lake	Leitura_atualização	10G	00:00:06	Q1.3	Presto

Delta Lake	Leitura_atualização	10G	00:00:08	Q2.1	Presto
Delta Lake	Leitura_atualização	10G	00:00:07	Q2.2	Presto
Delta Lake	Leitura_atualização	10G	00:00:07	Q2.3	Presto
Delta Lake	Leitura_atualização	10G	00:00:08	Q3.1	Presto
Delta Lake	Leitura_atualização	10G	00:00:07	Q3.2	Presto
Delta Lake	Leitura_atualização	10G	00:00:08	Q3.3	Presto
Delta Lake	Leitura_atualização	10G	00:00:10	Q3.4	Presto
Delta Lake	Leitura_atualização	10G	00:00:10	Q4.1	Presto
Delta Lake	Leitura_atualização	10G	00:00:10	Q4.2	Presto
Delta Lake	Leitura_atualização	10G	00:00:10	Q4.3	Presto

A.3.Consultas simples

```
select *
from delta_lineorder
```

```
select *
from hudi_lineorder
```

```
select *
from iceberg_lineorder
```

A.3.1. Após a escrita

tecnologia	Operação	Tamanho	Tempo	Query	Engine
Delta Lake	Leitura_escrita	1G	00:00:33		Athena
Apache Hudi	Leitura_escrita	1G	00:00:41		Athena
Apache Iceberg	Leitura_escrita	1G	00:00:21		Athena
Delta Lake	Leitura_escrita	10G	00:01:14		Athena
Apache Hudi	Leitura_escrita	10G	00:01:30		Athena
Apache Iceberg	Leitura_escrita	10G	00:01:29		Athena

A.3.2. Após a atualização

tecnologia	Operação	Tamanho	Tempo	Query	Engine
Delta Lake	Leitura_atualização	1G	00:00:56		Athena
Apache Hudi	Leitura_atualização	1G	00:00:55		Athena
Apache Iceberg	Leitura_atualização	1G	00:00:20		Athena
Delta Lake	Leitura_atualização	10G	00:01:49		Athena
Apache Hudi	Leitura_atualização	10G	00:01:31		Athena
Apache Iceberg	Leitura_atualização	10G	00:01:16		Athena